

SSV (Oracle - Off-Chain) - Critical Components

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Oracle	Documentation quality	Low	
Timeline	2026-03-23 through 2026-04-03	Test quality	Medium	
Language	Go	Total Findings	10	Fixed: 7 Acknowledged: 3
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Source Code	ssvlabs/ssv-oracle  #a67396e 	Medium severity findings ⓘ	1	Fixed: 1
Auditors	- Leonardo Passos Senior Research Engineer - Andy Lin Senior Auditing Engineer - Jennifer Wu Auditing Engineer	Low severity findings ⓘ	6	Fixed: 4 Acknowledged: 2
		Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	3	Fixed: 2 Acknowledged: 1

Summary of Findings

`ssv-oracle` is an off-chain Go service that publishes Merkle roots of SSV cluster effective balances to the SSV Network contract on Ethereum. It subscribes to beacon chain finality events, syncs SSV contract events from the execution layer, aggregates validator effective balances by cluster, builds a Merkle tree, and commits the root on-chain via `commitRoot`.

An optional **cluster updater** (`--updater` flag) runs alongside the oracle. It listens for `RootCommitted` events and calls `updateClusterBalance` for each cluster whose on-chain balance differs from the committed Merkle leaf, submitting a Merkle proof per cluster.

The assessment identified several issues in the SSV Oracle architecture that primarily impact consistency, reliability, and operational robustness. The most notable risk stems from shared state and concurrency patterns, where unfinalized or inconsistent data can be incorporated into oracle commits, potentially leading to incorrect Merkle roots and, in edge cases, quorum disruption. Additional findings highlight gaps in validation, error handling, and retry logic, which may cause avoidable transaction failures, stale state usage, or missed commit windows under transient conditions. There are also concurrency and infrastructure concerns, such as nonce collisions, lack of strict RPC security guarantees, and absence of exclusive database access controls. Overall, while most issues are low severity, collectively they indicate opportunities to strengthen determinism, isolation between components, and resilience against network or operational edge cases.

Fix review 1 (commit `c9e972e`)

The shared patches fixed 6 of the 10 reported issues; 1 issue was not fixed (**SSV-OR-1**), while 3 were acknowledged. In the latter case, the underlying concerns remain accepted by the client as operational assumptions or design trade-offs rather than as a technical issue.

All the patches addressing the fixed issues introduced concrete code-level changes, including root validation before processing, improved revert handling and visibility, rollback on ABI unpack failures, nonce serialization, and correct handling of stale or liquidated cluster cases.

Fix review 2 (commit `3c1f874`)

The shared patch fixed the **SSV-OR-1** issue by decoupling the updater head-state from the clusters table. The fix has been merged into the `main` branch at commit `ea59b07`. **No issues were left unfixed at this round.**

ID	DESCRIPTION	SEVERITY	STATUS
SSV-OR-1	Shared EventSyncer Allows Updater to Corrupt Oracle's Cluster State — Incorrect Merkle Root	Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
SSV-OR-2	Updater Does Not Verify Event Root Against Local Commit Root	• Low ⓘ	Fixed
SSV-OR-3	Updater Does Not Classify and Log Revert Reasons, Causing Wrong Skip/Abort Handling	• Low ⓘ	Fixed
SSV-OR-4	Syncer Advances Cursor on Handled-Event Parse Failures	• Low ⓘ	Fixed
SSV-OR-5	Validator Balance Fetch Uses Moving "finalized" Alias Instead of Pinned Slot	• Low ⓘ	Acknowledged
SSV-OR-6	Oracle and Updater Share TxManager without Mutex — Concurrent Sendtransaction() Causes Nonce Collision	• Low ⓘ	Fixed
SSV-OR-7	No Commit-Level Retry — Transient RPC Failure Permanently Skips Epoch, Can Break Quorum	• Low ⓘ	Acknowledged
SSV-OR-8	<code>ClusterIsLiquidated</code> Revert Silently Skipped Instead of Triggering State Refresh	• Informational ⓘ	Fixed
SSV-OR-9	Updater Wastes Cycles on Doomed Transactions After Root Rotation	• Informational ⓘ	Fixed
SSV-OR-10	No TLS Enforcement or Authentication on Execution/consensus RPC Connections	• Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.

- 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
- 3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
- 4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Phase 1 (this audit)

- eth/syncer/syncer.go
- eth/syncer/parser.go
- oracle/oracle.go
- merkle/tree.go
- contract/client.go
- updater/updater.go

Note: When investigating these, occasionally, we refer to other files if we see that their functionality affects the files in scope.

Repo: `https://github.com/ssvlabs/ssv-oracle`

Operational Considerations

- **Commit cycle.** The oracle subscribes to beacon finalized checkpoint events (SSE). On each checkpoint it syncs SSV contract events to the finalized block, fetches validator effective balances, builds a Merkle tree aggregated by cluster, and submits the root via `commitRoot`. The commit schedule is configurable as a multi-phase epoch-aligned interval.
- **Updater cycle.** When `--updater` is enabled, a second goroutine listens for `RootCommitted` events. For each cluster whose on-chain balance differs from the committed leaf, it submits an `updateClusterBalance` transaction with a Merkle proof.
- **Single process, shared state.** Oracle and updater run as goroutines inside one process, sharing a single SQLite database (WAL mode) and `EventSyncer` instance.
- **Trust in beacon and execution nodes.** The oracle trusts the beacon node for finalized checkpoint events and validator effective balances, and the execution RPC for contract event logs and transaction submission. Correctness of the committed root depends entirely on the integrity of these sources.
- **Permissioned commit, permissionless update.** `commitRoot` is restricted on-chain to authorized oracle addresses. `updateClusterBalance` is permissionless — anyone with a valid Merkle proof for a committed root can call it.
- **MEV protection.** The updater optionally broadcasts `updateClusterBalance` through MEV-protected RPCs (Flashbots, Titan) to prevent frontrunning of balance updates that could trigger liquidations. `commitRoot` does not use MEV protection (permissioned, no frontrun risk).

Key Actors And Their Capabilities

- **Oracle operator** Runs the `ssv-oracle` binary, signs and submits `commitRoot` transactions. The SSV Network contract restricts `commitRoot` to authorized oracle addresses.
- **Updater** (same binary, `--updater`) Calls `updateClusterBalance` with Merkle proofs after a root is committed. Anyone with a valid proof for a committed root can call it.
- **Beacon node** Provides finalized checkpoint SSE stream and validator effective balances. Trusted external dependency (read-only).
- **Execution RPC** Provides contract event logs, block data, chain ID, and transaction submission. Trusted external dependency.
- **MEV RPCs** Flashbots / Titan builder endpoints for frontrun-protected `updateClusterBalance` submission. Optional; used only by the updater.
- **SSV Network contract** Stores committed roots, enforces oracle permissions, processes cluster balance updates.

Findings

SSV-OR-1

Shared EventSyncer Allows Updater to Corrupt Oracle's Cluster State — Incorrect Merkle Root

• Medium ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `3c1f874`.

File(s) affected: `cmd/oracle/commands/run.go`, `eth/syncer/syncer.go`, `storage/storage.go`, `oracle/oracle.go`

Description: When the `--updater` flag is enabled, the oracle and updater run as concurrent goroutines against the same SQLite database.

The updater's `SyncClustersToHead` (`eth/syncer/syncer.go:400-440`) fetches events from the last finalized block all the way to the **unfinalized head**, then writes cluster state — including `is_active` — directly to the shared `clusters` table.

Meanwhile, the oracle's commit cycle reads from the same table. During `commit()`, after `SyncToBlock` pins contract events to the finalized block, `fetchClusterBalances` calls `GetActiveValidators` (`storage/storage.go:269-290`), which joins on `is_active = 1`.

If the updater processes an unfinalized `ClusterLiquidated` event between `SyncToBlock` and `GetActiveValidators`, it sets `is_active = 0` for that cluster. The oracle then excludes the cluster from the Merkle tree. Conversely, an unfinalized `ClusterReactivated` event can cause the oracle to spuriously *include* a cluster. In either case, the oracle commits a root based on an unfinalized state.

If a significant number of oracles face this situation, either the quorum can be compromised, or, worse, they can agree on an incorrect state.

Recommendation: Decouple the cluster state used by the oracle's commit cycle from the head-state writes performed by the updater. Options include maintaining a separate `clusters_head` table for updater-only consumption or caching head-state cluster data in memory inside the updater rather than in a shared table.

SSV-OR-2

Updater Does Not Verify Event Root Against Local Commit Root

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `00b16f67676b4e2255d151f1e77d1a8d85375f85`.
The client provided the following explanation:

Added a guard in `handleEvent` that compares `event.MerkleRoot` against `commit.MerkleRoot` before calling `processCommit`. If they differ (e.g., this oracle lost the commit race to another oracle), the event is skipped with a warning log. Prevents submitting doomed proofs built from local data that doesn't match the on-chain root.

File(s) affected: `updater/updater.go`

Description: `RootCommitted` event includes both `blockNum` and `merkleRoot`, but local commit lookup is keyed by block number only. Updater then rebuilds proofs from local commit data without first confirming `event.merkleRoot == localCommit.merkleRoot`.

If local data for that block differs from the on-chain committed root, updater can submit wrong proofs and repeatedly revert with `InvalidProof`.

Recommendation: Validate event root against local commit root before processing and skip tx submission for mismatches (with logging/metrics).

SSV-OR-3

Updater Does Not Classify and Log Revert Reasons, Causing Wrong Skip/Abort Handling

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `07ed3527f36ec3fd9151144ab816c36ee66bf942`.
The client provided the following explanation:

Replaced the catch-all if/else in `processAllClusters` with an explicit switch on revert reason, elevated recognized reverts from `Debug` to `Warn` with a reason field for operator visibility. This commit provides the switch structure; [SSV-OR-8](#) and [SSV-OR-9](#) add their specific cases in subsequent commits.

File(s) affected: `updater/updater.go`

Description: The updater collapses most `updateClusterBalance` reverts as `skipped` (except `IncorrectClusterState`) and records only coarse outcome counters (`updated`, `skipped`, `failed`) without reason labels.

This is problematic because these revert classes imply different actions: `MustUseLatestRoot` / `RootNotFound` are stale-batch signals, `IncorrectClusterState` is retry-after-resync, `UpdateTooFrequent` is defer-and-retry-later, and `InvalidProof` / `EBExceedsMaximum` / `EBBelowMinimum` are deterministic input/proof failures.

Collapsing them into `skipped` can waste gas on stale-root transactions, mis-handle stale/defer/retry/fail policy paths, and obscure root-cause visibility during operations.

Recommendation: Implement reason-to-action mapping and expose the same mapping in bounded `outcome + reason` metrics so each revert class follows the intended policy (batch-abort, retry/defer, stale-skip, or fail).

SSV-OR-4

Syncer Advances Cursor on Handled-Event Parse Failures

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `7edf09ac7f00a99333359bdf53215f64b5acd649` .

The client provided the following explanation:

processLog now returns an error (instead of nil) when a handled-event signature fails ABI unpack. The batch transaction rolls back and last_synced_block does not advance, preventing silent state drift. Unknown-event signatures (errUnknownEvent) retain the existing store-and-continue behavior. The error message includes block number, tx hash, and log index for operator diagnosis.

File(s) affected: `eth/syncer/parser.go` , `eth/syncer/syncer.go`

Description: For handled event types, parse failure is currently soft: the log is saved as "Unknown" and processing continues. The batch can still commit, and `last_synced_block` can still advance.

That means sync progress can move forward even though a required state transition was not applied locally. For cluster-affecting events, this creates silent drift between on-chain truth and off-chain DB state.

This is especially sensitive for active-state events used by Merkle/update flows:

- `ClusterLiquidated ( active=false )`
- `ClusterReactivated ( active=true )`
- `ClusterMigratedToETH ( active=true )`
- `ClusterBalanceUpdated` in auto-liquidation path

If one of these handled events is skipped due to parse failure, oracle may generate roots from incorrect active/membership inputs, and updater may submit using stale cluster fields and trigger avoidable reverts or skipped updates.

Recommendation: Consider implementing a degraded-sync safety mode for handled-event parse failures. If a handled event (for example `ClusterLiquidated` , `ClusterReactivated` , `ClusterMigratedToETH` , `ClusterBalanceUpdated`) fails to parse, record the failure (`block` , `logIndex` , `topic0` , `reason`), do not advance `last_synced_block` , and mark sync as degraded. While degraded, keep API/metrics running but block correctness-sensitive writes (`oracle.commitRoot` and updater `updateClusterBalance` submissions). Automatically retry replay from the failed block, and clear degraded mode only after replay succeeds.

SSV-OR-5

Validator Balance Fetch Uses Moving "finalized" Alias Instead of Pinned Slot

• Low ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We previously implemented state-root pinning (commit 0880815) and reverted it (commit d16b0ef) because non-archive beacon nodes cannot reliably serve state by any fixed identifier other than the "finalized" alias. Slot-based pinning has the same fundamental issue: if the epoch boundary slot was missed (no block proposed), the beacon must reconstruct the state from the nearest stored state, which is not guaranteed on non-archive Lighthouse, Prysm, Teku, Nimbus, or Lodestar configurations. The "finalized" alias is the only option that reliably hits the beacon's hot cache across all client implementations. The non-determinism window is narrow: finalization advances every ~12.8 minutes, while the concurrent balance fetch completes in seconds.

File(s) affected: `eth/beacon/client.go` , `oracle/oracle.go`

Description: `GetValidatorBalances` (`eth/beacon/client.go:176`) queries the beacon API with `State: "finalized"` , a moving alias that resolves to whatever epoch the beacon node considers finalized at request time. The oracle already guards against large drift via `VerifyFinalizedBlockRoot` (`oracle/oracle.go:162-164`), which fails the commit if finalization has advanced past the checkpoint epoch before the balance fetch begins. However, a narrow race remains: finalization can advance *after* the verify passes and *during* the batched balance fetch (up to `balanceFetchConcurrency` concurrent goroutines). In that case, some batches resolve against epoch `E` and others against `E+1` , producing a merkle root no other oracle can reproduce — splitting quorum.

Recommendation: Pin the balance fetch to an explicit slot derived from the checkpoint epoch:

```
slot := checkpoint.Epoch * SlotsPerEpoch // 32 on mainnet
stateID := strconv.FormatUint(slot, 10)
```

Pass `stateID` through `fetchClusterBalances` to `GetValidatorBalances` and use it as `api.ValidatorsOpts.State`. The beacon API accepts numeric slot identifiers; if the slot was skipped, the beacon returns the state of the most recent prior slot — deterministic across all go-routines when getting validator's balances.

SSV-OR-6

Oracle and Updater Share TxManager without Mutex — Concurrent `Sendtransaction()` Causes Nonce Collision

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `c55cf6f54255663c7dc0144876300547849e83`.

The client provided the following explanation:

Added `sync.Mutex` to `TxManager`, locked for the full `SendTransaction` lifecycle. Prevents nonce collision between concurrent oracle `CommitRoot` and updater `UpdateClusterBalance` calls. Removed the misplaced "Not thread-safe" comment from `contract/client.go`.

File(s) affected: `contract/client.go`

Description: `TxManager.SendTransaction()` has no mutex. It fetches the pending nonce via `PendingNonceAt()` at `manager.go:169` and reuses it across the retry loop. The `TxManager` struct has no `sync.Mutex` field.

Oracle and updater share a single `contract.Client` (and thus a single `TxManager`) and run in separate goroutines via `errgroup` in `run.go:213–238`. Oracle calls `Client.CommitRoot()` and updater calls `Client.UpdateClusterBalance()`, both of which delegate to `TxManager.SendTransaction()`. The comment at `client.go:174` acknowledges "Not thread-safe; callers must ensure sequential execution" but no caller-side serialization exists.

The race (*Time-Of-Check to Time-Of-Use* on nonce):

1. Goroutine A calls `PendingNonceAt()` → gets nonce N (tx not yet sent)
2. Goroutine B calls `PendingNonceAt()` → gets same nonce N (A hasn't reached mempool yet)
3. Both sign and send with nonce N — one gets `nonce too low`

If the race does hit, the failing call gets `nonce too low` and is retried on the next cycle. Worst case is one wasted commit window (~6.4 min) or one delayed cluster update.

Mitigating factors: `PendingNonceAt()` accounts for pending txs already in the mempool. The updater is event-driven and cannot begin processing until the oracle's `CommitRoot()` tx is mined and confirmed. The only realistic overlap is cross-cycle. Oracle commits are spaced ~6.4 min apart and `SendTransaction()` blocks until receipt.

Example:

1. Oracle reaches its next commit target and calls `Client.CommitRoot()` → enters `TxManager.SendTransaction()`
2. Concurrently, updater receives a `RootCommitted` event from the previous cycle and calls `Client.UpdateClusterBalance()` → enters `TxManager.SendTransaction()`
3. Both goroutines call `PendingNonceAt()` before either's tx reaches the mempool — both get nonce N
4. Both sign and submit with nonce N
5. The RPC accepts the first arrival; the second fails with `nonce too low`
6. The failed goroutine returns an error, and the operation is retried on the next cycle>

Recommendation: Add a `sync.Mutex` to `TxManager` that serializes the entire `SendTransaction()` body:

```
type TxManager struct {
    mu      sync.Mutex // serializes SendTransaction() calls
    client  *ethclient.Client
    // ...
}

func (m *TxManager) SendTransaction(ctx context.Context, opts *TxOpts) (*types.Receipt, error) {
    m.mu.Lock()
    defer m.mu.Unlock()
    // ... rest unchanged
}
```

This blocks the second caller for the duration of the first tx lifecycle (12 — 24sf or confirmation), which is $\leq$ given the 6.4 min cycle spacing.

SSV-OR-7

No Commit-Level Retry — Transient RPC Failure Permanently Skips Epoch, Can Break Quorum

• Low ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

SSV oracle operators are independent partners with independent infrastructure. The audit's quorum-break scenario (correlated failures from shared RPC/beacon) does not apply — if one oracle's RPC goes down, the other three oracles commit normally. The impact is limited to a single oracle missing one commit cycle, with automatic recovery on the next cycle. Per-step retries (eth.WithRetry: 4 attempts, about 14-17s total per call) already handle typical transient failures. A commit-level retry would add complexity (~30 lines including a freshness guard) for a scenario whose operational impact is minimal under the independent-operator model.

File(s) affected: `contract/client.go`

Description: The oracle's `Run` loop (`oracle/oracle.go:110–165`) has RPC-level retry per individual call (3 attempts, 2-15s exponential backoff via `eth.WithRetry`) but no commit-level retry. When `commit()` fails after RPC retries are exhausted, the oracle records `outcomeError`, advances `nextTarget = schedule.NextTarget(fullyFinalized)`, and the failed epoch is permanently abandoned.

The `commit()` function (`oracle/oracle.go:168–248`) calls 6 sequential RPC-dependent steps: `SyncToBlock`, `VerifyFinalizedBlockRoot`, `fetchClusterBalances`, `InsertPendingCommit`, `GetCommittedRoot`, and `CommitRoot`. Any single step failing after retries causes the entire commit to fail with no recovery path.

With a 225-epoch commit interval (~24 hours), a transient RPC failure lasting 40+ seconds (enough to exhaust 3 retries) causes the affected oracle to skip the epoch entirely. In a 3-of-4 oracle setup, if two oracles experience correlated failures (e.g., shared beacon node or RPC provider), both skip the epoch and the remaining two oracles cannot reach the 75% quorum threshold.

Example:

1. Oracle with commit interval = 225 epochs (~24h), `nextTarget` = epoch 450
2. Beacon node undergoes routine maintenance (~60s downtime)
3. Epoch 451 finalizes and the oracle receives the checkpoint via SSE.
4. Oracle calls `commit()` → `VerifyFinalizedBlockRoot` fails (connection refused). All 3 retry attempts exhausted over ~40s. `commit()` returns error.
5. `nextTarget` advances from 450 to 675. The beacon node recovers 20s later, but the epoch is already abandoned.
6. All checkpoints for epochs 451-674 are ignored. Cluster balance data remains stale for ~24h until epoch 675.

Recommendation: Consider adding a commit-level retry with backoff before advancing `nextTarget`. On failure, wait a short period (e.g., 30 seconds) and re-attempt `commit()` before giving up and advancing to the next epoch. Note that `InsertPendingCommit` must be idempotent or skipped on retry, since the pending record from the first attempt may still exist.

SSV-OR-8

`ClusterIsLiquidated` Revert Silently Skipped Instead of Triggering State Refresh

• Informational ⓘ Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `f6ec817c6c5a4e8848575a76f53c89e5c372d339`.

The client provided the following explanation:

`ClusterIsLiquidated` is now routed to the stale-cluster recovery path (`SyncClustersToHead` + retry) as a dedicated case in the [SSV-OR-3](#) switch. After `SyncClustersToHead` refreshes local `is_active`, the next oracle commit cycle correctly excludes the liquidated cluster from the merkle tree.

File(s) affected: `eth/syncer/syncer.go`

Description: When a cluster is liquidated between finalized sync points, the updater's local storage still marks it as active. The contract validates liquidation status before cluster state (`ClusterLib.validateClusterIsNotLiquidated()`) reverts with `ClusterIsLiquidated` before reaching the state hash check), so a liquidated cluster never hits `IncorrectClusterState`.

In `Updater.processAllClusters()#L242`, only `IncorrectClusterState` triggers the stale-cluster recovery path (`SyncClustersToHead`). The `ClusterIsLiquidated` revert falls into the catch-all else branch at `Updater.processAllClusters()#L245` (`stats.skipped++`), with no state refresh.

All oracle instances are affected identically. The cluster remains marked active in local storage, repeating the failed update cycle every commit until the next `SyncToBlock` call advances the event cursor past the liquidation event. With the default schedule interval of 225 epochs (~24 hours), the cluster is stuck in a silently-failing loop for up to a full commit cycle.

Impact is low because the cluster is already liquidated on-chain, so the balance update the updater is trying to perform is effectively moot. No gas is burned (reverts are caught by `estimateGas`), and the state self-heals on the next commit cycle's `SyncToBlock`.

Recommendation: Add `ClusterIsLiquidated` to the stale-state check in `Updater.processAllClusters()`#L242 so it triggers `SyncClustersToHead` alongside `IncorrectClusterState`:

```
if revertErr.Reason == "IncorrectClusterState" || revertErr.Reason == "ClusterIsLiquidated" {
    staleLeaves = append(staleLeaves, leaf)
}
```

SSV-OR-9

Updater Wastes Cycles on Doomed Transactions After Root Rotation

- Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `c9e972e039d6d5663754340710930fc94556b409`.

The client provided the following explanation:

MustUseLatestRoot and RootNotFound now return immediately from processAllClusters as dedicated cases in the [SSV-OR-3](#) switch. Returning nil staleLeaves ensures processCommit skips the stale-leaf recovery path for an obsolete batch. No on-chain gas is burned — these reverts are caught at estimateGas simulation time.

File(s) affected: `updater/updater.go`

Description: When the on-chain `latestCommittedBlock` advances (a newer `commitRoot` reaches quorum), every remaining `updateClusterBalance` call in the current batch reverts with `MustUseLatestRoot`. The updater does not detect this condition — it classifies `MustUseLatestRoot` as a generic skip (`updater/updater.go:241–249`) and continues iterating all remaining leaves, each of which reverts identically. This wastes $(N - k)$ cycles, where k is the number of leaves processed before the root rotated and N is total leaves.

Recommendation: Short-circuit the batch on the first `MustUseLatestRoot` revert — all subsequent leaves will fail identically.

SSV-OR-10

No TLS Enforcement or Authentication on Execution/consensus RPC Connections

- Informational ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

The oracle uses go-ethereum's ethclient.Dial with operator-configured URLs, following the same connection pattern used by all Ethereum clients (geth, prysm, lighthouse, teku). No Ethereum client or validator tooling enforces HTTPS on RPC URLs — TLS is an infrastructure concern managed by operators. Oracle operators are professional partners who run their own execution and beacon nodes, typically co-located or within private networks. The default API binding is 127.0.0.1 (localhost only). The audit's Engine API JWT recommendation is not applicable — this oracle uses regular JSON-RPC, not the Engine API.

File(s) affected: `contract/client.go`

Description: The oracle accepts both HTTP and HTTPS URLs for execution RPC (`eth_rpc`), beacon RPC (`beacon_url`), WebSocket (`eth_ws_rpc`), and MEV relay endpoints (`mev_rpcs`). The `validateURL` function in `cmd/oracle/commands/config.go:76–79` explicitly allows both `"http"` and `"https"` schemes:

```
validateURL(c.EthRPC, "eth_rpc", "http", "https")
validateURL(c.BeaconURL, "beacon_url", "http", "https")
```

The RPC connections in `contract/client.go:59` (`ethclient.Dial`) and `eth/beacon/client.go` use Go's default HTTP transport with no certificate pinning, mutual TLS, or authentication headers.

When an operator configures plain HTTP endpoints — especially over an untrusted network — all RPC traffic is transmitted in cleartext. This exposes the oracle to man-in-the-middle (MITM) attacks where an attacker can:

- Inject fabricated event logs
- Tamper with block data
- Feed stale or replayed responses
- Intercept signed transactions
- Manipulate validator performance data

Exploit Scenario:

1. Operator configures the oracle with `eth_rpc: http://10.0.0.5:8545` on a shared network.
2. Attacker gains a network position between the oracle and the RPC nodes (ARP spoofing, compromised switch, rogue cloud VM).
3. Attacker intercepts `eth_getLogs` responses and injects a fabricated `ClusterLiquidated` event.
4. The oracle processes the fake event as legitimate on-chain data.
5. The oracle acts on the fabricated state — potentially triggering incorrect liquidation reports or corrupting its local database.

Recommendation:

1. Enforce HTTPS/WSS in production: Change `validateURL` to reject `http` and `ws` schemes unless an explicit `--allow-insecure` flag is set.
2. Document the trust assumption: Clearly document that the oracle implicitly trusts all data from configured RPC endpoints.
3. Support JWT authentication: For self-hosted execution/consensus clients, support the Engine API JWT authentication standard.
4. Log a warning on HTTP: If insecure schemes are allowed, emit a prominent startup warning.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/ssvlabs/ssv-oracle`
- `bb6...2f1` `./.dockerignore`
 - `3ac...e3e` `./.env.example`
 - `75d...2db` `./.github/CODEOWNERS`
 - `f21...9ea` `./.github/renovate.json`
 - `d16...c00` `./.github/workflows/ci.yml`
 - `3c9...c18` `./.github/workflows/release.yml`
 - `9e0...ed5` `./.gitignore`
 - `4e8...f7e` `./CLAUDE.md`
 - `100...9f9` `./Dockerfile`

- 397...986 ./LICENSE
- ede...76e ./Makefile
- 629...05c ./README.md
- 932...9ae ./api/api.go
- cc0...c10 ./api/api_test.go
- 9b4...e84 ./api/handlers.go
- 16d...205 ./api/types.go
- 4f2...b16 ./api/ui/index.html
- 755...912 ./cmd/oracle/commands/config.go
- 61f...e94 ./cmd/oracle/commands/config_test.go
- 7f3...d9d ./cmd/oracle/commands/root.go
- f4b...91c ./cmd/oracle/commands/run.go
- b05...780 ./cmd/oracle/commands/version.go
- bda...03e ./cmd/oracle/main.go
- 849...f1c ./config.hoodi.yaml
- bc8...f4e ./config.mainnet.yaml
- 4e6...763 ./config.yaml.example
- 407...9f6 ./contract/SSVNetwork.abi
- 8ba...543 ./contract/SSVNetworkViews.abi
- 342...f6c ./contract/abi.go
- f30...4e6 ./contract/client.go
- 56c...65c ./contract/client_test.go
- 03f...6af ./contract/events.go
- 960...e28 ./docker-compose.yml
- 085...4e4 ./docs/api.md
- 3e2...0c8 ./docs/deployment.md
- e9a...279 ./eth/beacon/client.go
- acb...35a ./eth/beacon/client_test.go
- 9ed...796 ./eth/execution/batch_size.go
- 0e2...77d ./eth/execution/batch_size_test.go
- 8fb...7f3 ./eth/execution/client.go
- 5b9...dba ./eth/execution/client_test.go
- 997...b77 ./eth/retry.go
- dc1...f27 ./eth/retry_test.go
- 171...bcc ./eth/syncer/observability.go
- 5fc...353 ./eth/syncer/parser.go
- dab...a85 ./eth/syncer/parser_test.go
- 96e...ce2 ./eth/syncer/syncer.go
- 107...c99 ./eth/syncer/syncer_test.go
- 6f7...44a ./eth/syncer/topics.go
- 661...dc0 ./eth/syncer/topics_test.go
- b98...701 ./eth/syncer/types.go
- 422...abb ./eth/syncer/types_test.go
- b9a...8a0 ./go.mod
- 8c9...d58 ./go.sum
- 2ac...88a ./logger/logger.go
- c50...041 ./merkle/encoding.go
- 9fe...0b8 ./merkle/encoding_test.go
- 779...cc1 ./merkle/golden_test.go
- 53e...fe3 ./merkle/tree.go
- 5a1...b8e ./merkle/tree_test.go
- a02...7fe ./observability/observability.go
- d31...d37 ./oracle/observability.go
- 4a0...122 ./oracle/oracle.go
- 95b...f36 ./oracle/oracle_test.go

- 4ed...079 ./oracle/phase.go
- 087...8e1 ./oracle/phase_test.go
- 75d...f91 ./storage/schema.sql
- ac2...d2d ./storage/storage.go
- 832...9f2 ./storage/storage_test.go
- 3b1...7ab ./txmanager/config.go
- 99a...1b1 ./txmanager/config_test.go
- 7da...266 ./txmanager/manager.go
- 534...012 ./txmanager/manager_test.go
- 4bb...745 ./txmanager/observability.go
- ae6...06d ./updater/observability.go
- 7b2...78e ./updater/updater.go
- ed3...fe9 ./updater/updater_test.go
- 532...da8 ./wallet/env.go
- 8f6...e5d ./wallet/factory.go
- a27...de4 ./wallet/keystore.go
- 202...d3c ./wallet/signer.go
- 084...fa2 ./wallet/signer_test.go

Test Suite Results

```

=== RUN    TestHandleGetCommit_NoCommit
--- PASS: TestHandleGetCommit_NoCommit (0.00s)
=== RUN    TestHandleGetCommit_StorageError
2026-05-17T17:05:58.339-0400 ERROR logger/logger.go:110 Failed to get commit {"error": "get
latest commit: db connection lost"}
github.com/ssvlabs/ssv-oracle/logger.Errorw
github.com/ssvlabs/ssv-oracle/logger/logger.go:110
github.com/ssvlabs/ssv-oracle/api.(*Server).handleResolveError
github.com/ssvlabs/ssv-oracle/api/handlers.go:202
github.com/ssvlabs/ssv-oracle/api.(*Server).handleGetCommit
github.com/ssvlabs/ssv-oracle/api/handlers.go:28
github.com/ssvlabs/ssv-oracle/api.TestHandleGetCommit_StorageError
github.com/ssvlabs/ssv-oracle/api/api_test.go:78
testing.tRunner
testing/testing.go:1934
--- PASS: TestHandleGetCommit_StorageError (0.00s)
=== RUN    TestHandleGetCommit_InvalidEpoch
--- PASS: TestHandleGetCommit_InvalidEpoch (0.00s)
=== RUN    TestHandleGetCommit_Basic
--- PASS: TestHandleGetCommit_Basic (0.00s)
=== RUN    TestHandleGetCommit_FullTree
--- PASS: TestHandleGetCommit_FullTree (0.00s)
=== RUN    TestHandleGetCommit_ClusterInfoEnrichment
--- PASS: TestHandleGetCommit_ClusterInfoEnrichment (0.00s)
=== RUN    TestHandleGetCommit_SingleCluster
--- PASS: TestHandleGetCommit_SingleCluster (0.00s)
=== RUN    TestHandleGetProof_InvalidClusterID
=== RUN    TestHandleGetProof_InvalidClusterID/empty
=== RUN    TestHandleGetProof_InvalidClusterID/no_prefix
=== RUN    TestHandleGetProof_InvalidClusterID/too_short
=== RUN    TestHandleGetProof_InvalidClusterID/too_long
=== RUN    TestHandleGetProof_InvalidClusterID/invalid_hex
--- PASS: TestHandleGetProof_InvalidClusterID (0.00s)
--- PASS: TestHandleGetProof_InvalidClusterID/empty (0.00s)
--- PASS: TestHandleGetProof_InvalidClusterID/no_prefix (0.00s)
--- PASS: TestHandleGetProof_InvalidClusterID/too_short (0.00s)
--- PASS: TestHandleGetProof_InvalidClusterID/too_long (0.00s)
--- PASS: TestHandleGetProof_InvalidClusterID/invalid_hex (0.00s)
=== RUN    TestHandleGetProof_NoCommit
--- PASS: TestHandleGetProof_NoCommit (0.00s)
=== RUN    TestHandleGetProof_ClusterNotFound

```

```
--- PASS: TestHandleGetProof_ClusterNotFound (0.00s)
=== RUN    TestHandleGetProof_Success
--- PASS: TestHandleGetProof_Success (0.00s)
=== RUN    TestHandleGetProof_WithEpoch
=== RUN    TestHandleGetProof_WithEpoch/epoch_100_returns_proof_from_historical_commit
=== RUN    TestHandleGetProof_WithEpoch/no_epoch_returns_latest
--- PASS: TestHandleGetProof_WithEpoch (0.00s)
    --- PASS: TestHandleGetProof_WithEpoch/epoch_100_returns_proof_from_historical_commit (0.00s)
    --- PASS: TestHandleGetProof_WithEpoch/no_epoch_returns_latest (0.00s)
=== RUN    TestHandleGetProof_SingleCluster
--- PASS: TestHandleGetProof_SingleCluster (0.00s)
=== RUN    TestParseClusterID
=== RUN    TestParseClusterID/valid
=== RUN    TestParseClusterID/all_zeros
=== RUN    TestParseClusterID/all_ff
=== RUN    TestParseClusterID/empty
=== RUN    TestParseClusterID/no_prefix
=== RUN    TestParseClusterID/short
=== RUN    TestParseClusterID/long
=== RUN    TestParseClusterID/invalid_hex
=== RUN    TestParseClusterID/uppercase_valid
--- PASS: TestParseClusterID (0.00s)
    --- PASS: TestParseClusterID/valid (0.00s)
    --- PASS: TestParseClusterID/all_zeros (0.00s)
    --- PASS: TestParseClusterID/all_ff (0.00s)
    --- PASS: TestParseClusterID/empty (0.00s)
    --- PASS: TestParseClusterID/no_prefix (0.00s)
    --- PASS: TestParseClusterID/short (0.00s)
    --- PASS: TestParseClusterID/long (0.00s)
    --- PASS: TestParseClusterID/invalid_hex (0.00s)
    --- PASS: TestParseClusterID/uppercase_valid (0.00s)
=== RUN    TestToHex
=== RUN    TestToHex/empty
=== RUN    TestToHex/single_byte
=== RUN    TestToHex/multiple_bytes
=== RUN    TestToHex/32_bytes
--- PASS: TestToHex (0.00s)
    --- PASS: TestToHex/empty (0.00s)
    --- PASS: TestToHex/single_byte (0.00s)
    --- PASS: TestToHex/multiple_bytes (0.00s)
    --- PASS: TestToHex/32_bytes (0.00s)
=== RUN    TestContentTypeMiddleware
--- PASS: TestContentTypeMiddleware (0.00s)
=== RUN    TestRecoveryMiddleware
2026-05-17T17:05:58.344-0400 ERROR logger/logger.go:110      Panic recovered {"error": "test panic",
"path": "/" }
github.com/ssvlabs/ssv-oracle/logger.Errorw
    github.com/ssvlabs/ssv-oracle/logger/logger.go:110
github.com/ssvlabs/ssv-oracle/api.TestRecoveryMiddleware.(*Server).recoveryMiddleware.func2.1
    github.com/ssvlabs/ssv-oracle/api/api.go:100
runtime.gopanic
    runtime/panic.go:783
github.com/ssvlabs/ssv-oracle/api.TestRecoveryMiddleware.func1
    github.com/ssvlabs/ssv-oracle/api/api_test.go:547
net/http.HandlerFunc.ServeHTTP
    net/http/server.go:2322
github.com/ssvlabs/ssv-oracle/api.TestRecoveryMiddleware.(*Server).recoveryMiddleware.func2
    github.com/ssvlabs/ssv-oracle/api/api.go:105
net/http.HandlerFunc.ServeHTTP
    net/http/server.go:2322
github.com/ssvlabs/ssv-oracle/api.TestRecoveryMiddleware
    github.com/ssvlabs/ssv-oracle/api/api_test.go:554
testing.tRunner
    testing/testing.go:1934
--- PASS: TestRecoveryMiddleware (0.00s)
=== RUN    TestHandleGetCommit_OddLeafCount
--- PASS: TestHandleGetCommit_OddLeafCount (0.00s)
=== RUN    TestHandleGetProof_OddLeafCount
--- PASS: TestHandleGetProof_OddLeafCount (0.00s)
=== RUN    TestHandleGetCommit_RootMismatch
2026-05-17T17:05:58.345-0400 ERROR logger/logger.go:110      Merkle root mismatch      {"computed":
"0x87bc2f90f84d8bb467f711ecfe125bce93c0cee9ec20e6838285938d5b77742c", "stored":
```

```
"0xff0000000000000000000000000000000000000000000000000000000000000000"}
github.com/ssvlabs/ssv-oracle/logger.Errorw
    github.com/ssvlabs/ssv-oracle/logger/logger.go:110
github.com/ssvlabs/ssv-oracle/api.(*Server).handleGetCommit
    github.com/ssvlabs/ssv-oracle/api/handlers.go:46
github.com/ssvlabs/ssv-oracle/api.TestHandleGetCommit_RootMismatch
    github.com/ssvlabs/ssv-oracle/api/api_test.go:681
testing.tRunner
    testing/testing.go:1934
--- PASS: TestHandleGetCommit_RootMismatch (0.00s)
=== RUN    TestHandleGetProof_RootMismatch
2026-05-17T17:05:58.345-0400    ERROR    logger/logger.go:110    Merkle root mismatch    {"computed":
"0x87bc2f90f84d8bb467f711ecfe125bce93c0cee9ec20e6838285938d5b77742c", "stored":
"0xff0000000000000000000000000000000000000000000000000000000000000000"}
github.com/ssvlabs/ssv-oracle/logger.Errorw
    github.com/ssvlabs/ssv-oracle/logger/logger.go:110
github.com/ssvlabs/ssv-oracle/api.(*Server).handleGetProof
    github.com/ssvlabs/ssv-oracle/api/handlers.go:121
github.com/ssvlabs/ssv-oracle/api.TestHandleGetProof_RootMismatch
    github.com/ssvlabs/ssv-oracle/api/api_test.go:715
testing.tRunner
    testing/testing.go:1934
--- PASS: TestHandleGetProof_RootMismatch (0.00s)
=== RUN    TestHandleGetCommit_EPOCHParam
=== RUN    TestHandleGetCommit_EPOCHParam/specific_epoch
=== RUN    TestHandleGetCommit_EPOCHParam/latest_has_prev
=== RUN    TestHandleGetCommit_EPOCHParam/nonexistent_epoch
=== RUN    TestHandleGetCommit_EPOCHParam/invalid_epoch
--- PASS: TestHandleGetCommit_EPOCHParam (0.00s)
    --- PASS: TestHandleGetCommit_EPOCHParam/specific_epoch (0.00s)
    --- PASS: TestHandleGetCommit_EPOCHParam/latest_has_prev (0.00s)
    --- PASS: TestHandleGetCommit_EPOCHParam/nonexistent_epoch (0.00s)
    --- PASS: TestHandleGetCommit_EPOCHParam/invalid_epoch (0.00s)
=== RUN    TestHandleGetCommit_TotalBalanceAndDiff
--- PASS: TestHandleGetCommit_TotalBalanceAndDiff (0.00s)
=== RUN    TestHandleGetCommit_FirstCommitNoDiff
--- PASS: TestHandleGetCommit_FirstCommitNoDiff (0.00s)
=== RUN    TestComputeDiff
--- PASS: TestComputeDiff (0.00s)
=== RUN    TestComputeDiff_NoChanges
--- PASS: TestComputeDiff_NoChanges (0.00s)
=== RUN    TestComputeDiff_OnlyAddedRemoved
--- PASS: TestComputeDiff_OnlyAddedRemoved (0.00s)
=== RUN    TestToHexOrEmpty
=== RUN    TestToHexOrEmpty/nil
=== RUN    TestToHexOrEmpty/empty
=== RUN    TestToHexOrEmpty/single_byte
=== RUN    TestToHexOrEmpty/32_bytes
--- PASS: TestToHexOrEmpty (0.00s)
    --- PASS: TestToHexOrEmpty/nil (0.00s)
    --- PASS: TestToHexOrEmpty/empty (0.00s)
    --- PASS: TestToHexOrEmpty/single_byte (0.00s)
    --- PASS: TestToHexOrEmpty/32_bytes (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/api    1.636s
?      github.com/ssvlabs/ssv-oracle/cmd/oracle    [no test files]
=== RUN    TestConfig_Validate
=== RUN    TestConfig_Validate/valid_config
=== RUN    TestConfig_Validate/whitespace_trimmed
=== RUN    TestConfig_Validate/missing_eth_rpc
=== RUN    TestConfig_Validate/invalid_eth_rpc_url
=== RUN    TestConfig_Validate/wrong_eth_rpc_scheme
=== RUN    TestConfig_Validate/missing_beacon_url
=== RUN    TestConfig_Validate/missing_ssv_contract
=== RUN    TestConfig_Validate/invalid_ssv_contract_address
=== RUN    TestConfig_Validate/zero_ssv_contract_address
=== RUN    TestConfig_Validate/missing_ssv_contract_deploy_block
=== RUN    TestConfig_Validate/empty_schedule
=== RUN    TestConfig_Validate/missing_ssv_views_contract
=== RUN    TestConfig_Validate/invalid_ssv_views_contract_address
=== RUN    TestConfig_Validate/updater:_missing_eth_ws_rpc
=== RUN    TestConfig_Validate/updater:_valid_config
```

```
=== RUN    TestConfig_Validate/wrong_eth_ws_rpc_scheme
=== RUN    TestConfig_Validate/optional_eth_ws_rpc_ignored_when_not_updater
=== RUN    TestConfig_Validate/multiple_errors_collected
=== RUN    TestConfig_Validate/duplicate_mev_rpcs
=== RUN    TestConfig_Validate/duplicate_mev_rpcs_with_spaces
=== RUN    TestConfig_Validate/valid_unique_mev_rpcs
--- PASS: TestConfig_Validate (0.00s)
    --- PASS: TestConfig_Validate/valid_config (0.00s)
    --- PASS: TestConfig_Validate/whitespace_trimmed (0.00s)
    --- PASS: TestConfig_Validate/missing_eth_rpc (0.00s)
    --- PASS: TestConfig_Validate/invalid_eth_rpc_url (0.00s)
    --- PASS: TestConfig_Validate/wrong_eth_rpc_scheme (0.00s)
    --- PASS: TestConfig_Validate/missing_beacon_url (0.00s)
    --- PASS: TestConfig_Validate/missing_ssv_contract (0.00s)
    --- PASS: TestConfig_Validate/invalid_ssv_contract_address (0.00s)
    --- PASS: TestConfig_Validate/zero_ssv_contract_address (0.00s)
    --- PASS: TestConfig_Validate/missing_ssv_contract_deploy_block (0.00s)
    --- PASS: TestConfig_Validate/empty_schedule (0.00s)
    --- PASS: TestConfig_Validate/missing_ssv_views_contract (0.00s)
    --- PASS: TestConfig_Validate/invalid_ssv_views_contract_address (0.00s)
    --- PASS: TestConfig_Validate/updater:_missing_eth_ws_rpc (0.00s)
    --- PASS: TestConfig_Validate/updater:_valid_config (0.00s)
    --- PASS: TestConfig_Validate/updater:_wrong_eth_ws_rpc_scheme (0.00s)
    --- PASS: TestConfig_Validate/optional_eth_ws_rpc_ignored_when_not_updater (0.00s)
    --- PASS: TestConfig_Validate/multiple_errors_collected (0.00s)
    --- PASS: TestConfig_Validate/duplicate_mev_rpcs (0.00s)
    --- PASS: TestConfig_Validate/duplicate_mev_rpcs_with_spaces (0.00s)
    --- PASS: TestConfig_Validate/valid_unique_mev_rpcs (0.00s)
=== RUN    TestConfig_Defaults
--- PASS: TestConfig_Defaults (0.00s)
=== RUN    TestConfig_MultipleErrors
--- PASS: TestConfig_MultipleErrors (0.00s)
=== RUN    TestLoadConfig_FileNotFound
--- PASS: TestLoadConfig_FileNotFound (0.00s)
=== RUN    TestLoadConfig_UnknownField
--- PASS: TestLoadConfig_UnknownField (0.00s)
=== RUN    TestLoadConfig_DefaultDBPath
--- PASS: TestLoadConfig_DefaultDBPath (0.00s)
=== RUN    TestLoadConfig_ExplicitDBPath
--- PASS: TestLoadConfig_ExplicitDBPath (0.00s)
=== RUN    TestLoadConfig_InvalidYAML
--- PASS: TestLoadConfig_InvalidYAML (0.00s)
=== RUN    TestLoadConfig_ValidationError
--- PASS: TestLoadConfig_ValidationError (0.00s)
=== RUN    TestValidateURL
=== RUN    TestValidateURL/valid_http
=== RUN    TestValidateURL/valid_https
=== RUN    TestValidateURL/valid_ws
=== RUN    TestValidateURL/valid_wss
=== RUN    TestValidateURL/empty
=== RUN    TestValidateURL/no_scheme
=== RUN    TestValidateURL/wrong_scheme
=== RUN    TestValidateURL/no_host
--- PASS: TestValidateURL (0.00s)
    --- PASS: TestValidateURL/valid_http (0.00s)
    --- PASS: TestValidateURL/valid_https (0.00s)
    --- PASS: TestValidateURL/valid_ws (0.00s)
    --- PASS: TestValidateURL/valid_wss (0.00s)
    --- PASS: TestValidateURL/empty (0.00s)
    --- PASS: TestValidateURL/no_scheme (0.00s)
    --- PASS: TestValidateURL/wrong_scheme (0.00s)
    --- PASS: TestValidateURL/no_host (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/cmd/oracle/commands 2.561s
=== RUN    TestSSVNetworkABI_Loaded
    client_test.go:31: SSVNetwork ABI loaded successfully (67017 bytes, 67 methods)
--- PASS: TestSSVNetworkABI_Loaded (0.00s)
=== RUN    TestSSVNetworkABI_Events
=== RUN    TestSSVNetworkABI_Events/RootCommitted
=== RUN    TestSSVNetworkABI_Events/ValidatorAdded
=== RUN    TestSSVNetworkABI_Events/ValidatorRemoved
=== RUN    TestSSVNetworkABI_Events/ClusterLiquidated
```

```

=== RUN    TestSSVNetworkABI_Events/ClusterReactivated
=== RUN    TestSSVNetworkABI_Events/ClusterWithdrawn
=== RUN    TestSSVNetworkABI_Events/ClusterDeposited
=== RUN    TestSSVNetworkABI_Events/ClusterMigratedToETH
=== RUN    TestSSVNetworkABI_Events/ClusterBalanceUpdated
--- PASS: TestSSVNetworkABI_Events (0.00s)
    --- PASS: TestSSVNetworkABI_Events/RootCommitted (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ValidatorAdded (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ValidatorRemoved (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterLiquidated (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterReactivated (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterWithdrawn (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterDeposited (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterMigratedToETH (0.00s)
    --- PASS: TestSSVNetworkABI_Events/ClusterBalanceUpdated (0.00s)
=== RUN    TestSSVNetworkABI_RootCommittedSignature
--- PASS: TestSSVNetworkABI_RootCommittedSignature (0.00s)
=== RUN    TestCluster
    client_test.go:132: Cluster: validatorCount=10, active=true
--- PASS: TestCluster (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/contract 2.166s
=== RUN    TestWithRetry_Success
--- PASS: TestWithRetry_Success (0.00s)
=== RUN    TestWithRetry_SuccessAfterRetries
2026-05-17T17:06:02.151-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "transient error"}
2026-05-17T17:06:02.153-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "transient error"}
--- PASS: TestWithRetry_SuccessAfterRetries (0.01s)
=== RUN    TestWithRetry_AllAttemptsFail
2026-05-17T17:06:02.156-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "persistent error"}
2026-05-17T17:06:02.157-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "persistent error"}
2026-05-17T17:06:02.160-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 3,
"maxAttempts": 4, "delay": "4ms", "error": "persistent error"}
--- PASS: TestWithRetry_AllAttemptsFail (0.01s)
=== RUN    TestWithRetry_ZeroMaxRetries
--- PASS: TestWithRetry_ZeroMaxRetries (0.00s)
=== RUN    TestWithRetry_ContextCanceled
2026-05-17T17:06:02.165-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 11, "delay": "1.234s", "error": "always fail"}
--- PASS: TestWithRetry_ContextCanceled (0.01s)
=== RUN    TestWithRetry_OneRetry
2026-05-17T17:06:02.175-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 2, "delay": "1ms", "error": "fail"}
--- PASS: TestWithRetry_OneRetry (0.00s)
=== RUN    TestWithRetry_NonRetriableErrors
=== RUN    TestWithRetry_NonRetriableErrors/400_Bad_Request
=== RUN    TestWithRetry_NonRetriableErrors/404_Not_Found
=== RUN    TestWithRetry_NonRetriableErrors/414_URI_Too_Long
=== RUN    TestWithRetry_NonRetriableErrors/429_Too_Many_Requests
2026-05-17T17:06:02.177-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "POST failed with status 429: {\"code\":429,\"message\": \"test
error\"}"}
2026-05-17T17:06:02.179-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "POST failed with status 429: {\"code\":429,\"message\": \"test
error\"}"}
2026-05-17T17:06:02.181-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 3,
"maxAttempts": 4, "delay": "4ms", "error": "POST failed with status 429: {\"code\":429,\"message\": \"test
error\"}"}
=== RUN    TestWithRetry_NonRetriableErrors/500_Internal_Server_Error
2026-05-17T17:06:02.186-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "POST failed with status 500: {\"code\":500,\"message\": \"test
error\"}"}
2026-05-17T17:06:02.188-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "POST failed with status 500: {\"code\":500,\"message\": \"test
error\"}"}
2026-05-17T17:06:02.190-0400 DEBUG logger/logger.go:83 Retrying after error {"attempt": 3,
"maxAttempts": 4, "delay": "5ms", "error": "POST failed with status 500: {\"code\":500,\"message\": \"test
error\"}"}

```

```
=== RUN    TestWithRetry_NonRetriableErrors/502_Bad_Gateway
2026-05-17T17:06:02.196-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "POST failed with status 502: {\\"code\\":502,\\"message\\":\\"test
error\\"}"}}
2026-05-17T17:06:02.197-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "POST failed with status 502: {\\"code\\":502,\\"message\\":\\"test
error\\"}"}}
2026-05-17T17:06:02.200-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 3,
"maxAttempts": 4, "delay": "5ms", "error": "POST failed with status 502: {\\"code\\":502,\\"message\\":\\"test
error\\"}"}}
=== RUN    TestWithRetry_NonRetriableErrors/503_Service_Unavailable
2026-05-17T17:06:02.206-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 1,
"maxAttempts": 4, "delay": "1ms", "error": "POST failed with status 503: {\\"code\\":503,\\"message\\":\\"test
error\\"}"}}
2026-05-17T17:06:02.207-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 2,
"maxAttempts": 4, "delay": "2ms", "error": "POST failed with status 503: {\\"code\\":503,\\"message\\":\\"test
error\\"}"}}
2026-05-17T17:06:02.210-0400    DEBUG    logger/logger.go:83 Retrying after error    {"attempt": 3,
"maxAttempts": 4, "delay": "5ms", "error": "POST failed with status 503: {\\"code\\":503,\\"message\\":\\"test
error\\"}"}}
--- PASS: TestWithRetry_NonRetriableErrors (0.04s)
    --- PASS: TestWithRetry_NonRetriableErrors/400_Bad_Request (0.00s)
    --- PASS: TestWithRetry_NonRetriableErrors/404_Not_Found (0.00s)
    --- PASS: TestWithRetry_NonRetriableErrors/414_URI_Too_Long (0.00s)
    --- PASS: TestWithRetry_NonRetriableErrors/429_Too_Many_Requests (0.01s)
    --- PASS: TestWithRetry_NonRetriableErrors/500_Internal_Server_Error (0.01s)
    --- PASS: TestWithRetry_NonRetriableErrors/502_Bad_Gateway (0.01s)
    --- PASS: TestWithRetry_NonRetriableErrors/503_Service_Unavailable (0.01s)
=== RUN    TestWithRetry_WrappedNonRetriableError
--- PASS: TestWithRetry_WrappedNonRetriableError (0.00s)
PASS
ok        github.com/ssvlabs/ssv-oracle/eth    1.865s
=== RUN    TestIsRetriable
=== RUN    TestIsRetriable/nil_error
=== RUN    TestIsRetriable/generic_error
=== RUN    TestIsRetriable/400_Bad_Request
=== RUN    TestIsRetriable/404_Not_Found
=== RUN    TestIsRetriable/429_Too_Many_Requests
=== RUN    TestIsRetriable/500_Internal_Server_Error
=== RUN    TestIsRetriable/502_Bad_Gateway
=== RUN    TestIsRetriable/503_Service_Unavailable
=== RUN    TestIsRetriable/wrapped_404
=== RUN    TestIsRetriable/wrapped_500
--- PASS: TestIsRetriable (0.00s)
    --- PASS: TestIsRetriable/nil_error (0.00s)
    --- PASS: TestIsRetriable/generic_error (0.00s)
    --- PASS: TestIsRetriable/400_Bad_Request (0.00s)
    --- PASS: TestIsRetriable/404_Not_Found (0.00s)
    --- PASS: TestIsRetriable/429_Too_Many_Requests (0.00s)
    --- PASS: TestIsRetriable/500_Internal_Server_Error (0.00s)
    --- PASS: TestIsRetriable/502_Bad_Gateway (0.00s)
    --- PASS: TestIsRetriable/503_Service_Unavailable (0.00s)
    --- PASS: TestIsRetriable/wrapped_404 (0.00s)
    --- PASS: TestIsRetriable/wrapped_500 (0.00s)
PASS
ok        github.com/ssvlabs/ssv-oracle/eth/beacon    2.253s
=== RUN    TestAdaptiveBatchSize_Decrease
=== RUN    TestAdaptiveBatchSize_Decrease/halve_once
=== RUN    TestAdaptiveBatchSize_Decrease/halve_twice
=== RUN    TestAdaptiveBatchSize_Decrease/halve_to_min
=== RUN    TestAdaptiveBatchSize_Decrease/already_at_min
--- PASS: TestAdaptiveBatchSize_Decrease (0.00s)
    --- PASS: TestAdaptiveBatchSize_Decrease/halve_once (0.00s)
    --- PASS: TestAdaptiveBatchSize_Decrease/halve_twice (0.00s)
    --- PASS: TestAdaptiveBatchSize_Decrease/halve_to_min (0.00s)
    --- PASS: TestAdaptiveBatchSize_Decrease/already_at_min (0.00s)
=== RUN    TestAdaptiveBatchSize_Increase
=== RUN    TestAdaptiveBatchSize_Increase/increase_once_from_100
=== RUN    TestAdaptiveBatchSize_Increase/increase_once_from_500
=== RUN    TestAdaptiveBatchSize_Increase/increase_once_from_900
=== RUN    TestAdaptiveBatchSize_Increase/increase_twice
=== RUN    TestAdaptiveBatchSize_Increase/increase_three_times
```

```
=== RUN    TestAdaptiveBatchSize_Increase/increase_capped_at_max
=== RUN    TestAdaptiveBatchSize_Increase/increase_to_max_over_multiple
=== RUN    TestAdaptiveBatchSize_Increase/already_at_max
=== RUN    TestAdaptiveBatchSize_Increase/large_value_caps_increment
--- PASS: TestAdaptiveBatchSize_Increase (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_once_from_100 (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_once_from_500 (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_once_from_900 (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_twice (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_three_times (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_capped_at_max (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/increase_to_max_over_multiple (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/already_at_max (0.00s)
    --- PASS: TestAdaptiveBatchSize_Increase/large_value_caps_increment (0.00s)
=== RUN    TestAdaptiveBatchSize_AIMD
--- PASS: TestAdaptiveBatchSize_AIMD (0.00s)
=== RUN    TestAdaptiveBatchSize_Defaults
--- PASS: TestAdaptiveBatchSize_Defaults (0.00s)
=== RUN    TestAdaptiveBatchSize_MinGreaterThanMax
--- PASS: TestAdaptiveBatchSize_MinGreaterThanMax (0.00s)
=== RUN    TestCategorizeBatchError
=== RUN    TestCategorizeBatchError/nil_error
=== RUN    TestCategorizeBatchError/block_range
=== RUN    TestCategorizeBatchError/exceed
=== RUN    TestCategorizeBatchError/query_returned_more
=== RUN    TestCategorizeBatchError/rate_limited
=== RUN    TestCategorizeBatchError/too_many_requests
=== RUN    TestCategorizeBatchError/429
=== RUN    TestCategorizeBatchError/payload_too_large
=== RUN    TestCategorizeBatchError/413
=== RUN    TestCategorizeBatchError/timeout
=== RUN    TestCategorizeBatchError/context_deadline
=== RUN    TestCategorizeBatchError/generic_limit
=== RUN    TestCategorizeBatchError/unrelated_error
--- PASS: TestCategorizeBatchError (0.00s)
    --- PASS: TestCategorizeBatchError/nil_error (0.00s)
    --- PASS: TestCategorizeBatchError/block_range (0.00s)
    --- PASS: TestCategorizeBatchError/exceed (0.00s)
    --- PASS: TestCategorizeBatchError/query_returned_more (0.00s)
    --- PASS: TestCategorizeBatchError/rate_limited (0.00s)
    --- PASS: TestCategorizeBatchError/too_many_requests (0.00s)
    --- PASS: TestCategorizeBatchError/429 (0.00s)
    --- PASS: TestCategorizeBatchError/payload_too_large (0.00s)
    --- PASS: TestCategorizeBatchError/413 (0.00s)
    --- PASS: TestCategorizeBatchError/timeout (0.00s)
    --- PASS: TestCategorizeBatchError/context_deadline (0.00s)
    --- PASS: TestCategorizeBatchError/generic_limit (0.00s)
    --- PASS: TestCategorizeBatchError/unrelated_error (0.00s)
=== RUN    TestGroupLogsByBlock
=== RUN    TestGroupLogsByBlock/empty_logs
=== RUN    TestGroupLogsByBlock/single_log
=== RUN    TestGroupLogsByBlock/sorts_by_block,_tx,_log_index
=== RUN    TestGroupLogsByBlock/groups_by_block
=== RUN    TestGroupLogsByBlock/error_on_missing_block_time
--- PASS: TestGroupLogsByBlock (0.00s)
    --- PASS: TestGroupLogsByBlock/empty_logs (0.00s)
    --- PASS: TestGroupLogsByBlock/single_log (0.00s)
    --- PASS: TestGroupLogsByBlock/sorts_by_block,_tx,_log_index (0.00s)
    --- PASS: TestGroupLogsByBlock/groups_by_block (0.00s)
    --- PASS: TestGroupLogsByBlock/error_on_missing_block_time (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/eth/execution 2.288s
=== RUN    TestApplyClusterEventToOverlay_InSet_AppliesPostEventState
--- PASS: TestApplyClusterEventToOverlay_InSet_AppliesPostEventState (0.00s)
=== RUN    TestApplyClusterEventToOverlay_OutOfSet_DoesNotPolluteOverlay
--- PASS: TestApplyClusterEventToOverlay_OutOfSet_DoesNotPolluteOverlay (0.00s)
=== RUN    TestApplyClusterEventToOverlay_MultipleEventsSameCluster_LastWins
--- PASS: TestApplyClusterEventToOverlay_MultipleEventsSameCluster_LastWins (0.00s)
=== RUN    TestNewParser
--- PASS: TestNewParser (0.00s)
=== RUN    TestParseLog_EmptyTopics
--- PASS: TestParseLog_EmptyTopics (0.00s)
```

```
=== RUN    TestParseLog_UnknownSignature
--- PASS: TestParseLog_UnknownSignature (0.00s)
=== RUN    TestParseLog_MissingOwnerTopic
--- PASS: TestParseLog_MissingOwnerTopic (0.00s)
=== RUN    TestEventSignatures
--- PASS: TestEventSignatures (0.00s)
=== RUN    TestEventSignatures_MatchABI
=== RUN    TestEventSignatures_MatchABI/ValidatorAdded
=== RUN    TestEventSignatures_MatchABI/ValidatorRemoved
=== RUN    TestEventSignatures_MatchABI/ClusterLiquidated
=== RUN    TestEventSignatures_MatchABI/ClusterReactivated
=== RUN    TestEventSignatures_MatchABI/ClusterWithdrawn
=== RUN    TestEventSignatures_MatchABI/ClusterDeposited
=== RUN    TestEventSignatures_MatchABI/ClusterMigratedToETH
=== RUN    TestEventSignatures_MatchABI/ClusterBalanceUpdated
--- PASS: TestEventSignatures_MatchABI (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ValidatorAdded (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ValidatorRemoved (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterLiquidated (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterReactivated (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterWithdrawn (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterDeposited (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterMigratedToETH (0.00s)
    --- PASS: TestEventSignatures_MatchABI/ClusterBalanceUpdated (0.00s)
=== RUN    TestNew
--- PASS: TestNew (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ValidatorAdded
--- PASS: TestComputeClusterIDFromEvent_ValidatorAdded (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ValidatorRemoved
--- PASS: TestComputeClusterIDFromEvent_ValidatorRemoved (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ClusterLiquidated
--- PASS: TestComputeClusterIDFromEvent_ClusterLiquidated (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ClusterReactivated
--- PASS: TestComputeClusterIDFromEvent_ClusterReactivated (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ClusterWithdrawn
--- PASS: TestComputeClusterIDFromEvent_ClusterWithdrawn (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ClusterDeposited
--- PASS: TestComputeClusterIDFromEvent_ClusterDeposited (0.00s)
=== RUN    TestComputeClusterIDFromEvent_ClusterMigratedToETH
--- PASS: TestComputeClusterIDFromEvent_ClusterMigratedToETH (0.00s)
=== RUN    TestClusterBalanceUpdatedEvent_ImplementsClusterEvent
--- PASS: TestClusterBalanceUpdatedEvent_ImplementsClusterEvent (0.00s)
=== RUN    TestComputeClusterIDFromEvent_UnknownType
--- PASS: TestComputeClusterIDFromEvent_UnknownType (0.00s)
=== RUN    TestClusterKey_AllEventTypes
=== RUN    TestClusterKey_AllEventTypes/ValidatorAdded
=== RUN    TestClusterKey_AllEventTypes/ValidatorRemoved
=== RUN    TestClusterKey_AllEventTypes/ClusterLiquidated
=== RUN    TestClusterKey_AllEventTypes/ClusterReactivated
=== RUN    TestClusterKey_AllEventTypes/ClusterWithdrawn
=== RUN    TestClusterKey_AllEventTypes/ClusterDeposited
=== RUN    TestClusterKey_AllEventTypes/ClusterMigratedToETH
--- PASS: TestClusterKey_AllEventTypes (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ValidatorAdded (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ValidatorRemoved (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ClusterLiquidated (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ClusterReactivated (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ClusterWithdrawn (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ClusterDeposited (0.00s)
    --- PASS: TestClusterKey_AllEventTypes/ClusterMigratedToETH (0.00s)
=== RUN    TestSameClusterIDForSameInputs
--- PASS: TestSameClusterIDForSameInputs (0.00s)
=== RUN    TestDifferentClusterIDForDifferentOwners
--- PASS: TestDifferentClusterIDForDifferentOwners (0.00s)
=== RUN    TestDifferentClusterIDForDifferentOperators
--- PASS: TestDifferentClusterIDForDifferentOperators (0.00s)
=== RUN    TestEventTopics_ReturnsCorrectCount
--- PASS: TestEventTopics_ReturnsCorrectCount (0.00s)
=== RUN    TestEventTopics_MatchesABI
--- PASS: TestEventTopics_MatchesABI (0.00s)
=== RUN    TestEventTopics_MatchesManualSignatures
--- PASS: TestEventTopics_MatchesManualSignatures (0.00s)
```

```
=== RUN    TestEventTopics_AllHandledEventsHaveHandlers
--- PASS: TestEventTopics_AllHandledEventsHaveHandlers (0.00s)
=== RUN    TestEventTopics_PanicsOnUnknownEvent
--- PASS: TestEventTopics_PanicsOnUnknownEvent (0.00s)
=== RUN    TestComputeClusterID
types_test.go:42: cluster ID: 0xb183c42279b4dc3eb381352db3458ae66a3439765e4f880a027f62ac2c4edba9
--- PASS: TestComputeClusterID (0.00s)
=== RUN    TestComputeClusterID_SortingInvariant
types_test.go:64: cluster ID (sorted):
0xb183c42279b4dc3eb381352db3458ae66a3439765e4f880a027f62ac2c4edba9
--- PASS: TestComputeClusterID_SortingInvariant (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/eth/syncer    1.773s
?      github.com/ssvlabs/ssv-oracle/logger        [no test files]
=== RUN    TestHashLeaf
--- PASS: TestHashLeaf (0.00s)
=== RUN    TestGoldenVector_OpenZeppelin
--- PASS: TestGoldenVector_OpenZeppelin (0.00s)
=== RUN    TestGoldenVector_EmptyTree
--- PASS: TestGoldenVector_EmptyTree (0.00s)
=== RUN    TestGoldenVector_SingleLeaf
--- PASS: TestGoldenVector_SingleLeaf (0.00s)
=== RUN    TestGoldenVector_ThreeLeaves
--- PASS: TestGoldenVector_ThreeLeaves (0.00s)
=== RUN    TestNewTree_Empty
--- PASS: TestNewTree_Empty (0.00s)
=== RUN    TestNewTree_SingleCluster
--- PASS: TestNewTree_SingleCluster (0.00s)
=== RUN    TestNewTree_Deterministic
--- PASS: TestNewTree_Deterministic (0.00s)
=== RUN    TestNewTree_VariousSizes
=== RUN    TestNewTree_VariousSizes/#00
=== RUN    TestNewTree_VariousSizes/#01
=== RUN    TestNewTree_VariousSizes/#02
=== RUN    TestNewTree_VariousSizes/#03
=== RUN    TestNewTree_VariousSizes/#04
=== RUN    TestNewTree_VariousSizes/#05
=== RUN    TestNewTree_VariousSizes/#06
--- PASS: TestNewTree_VariousSizes (0.00s)
--- PASS: TestNewTree_VariousSizes/#00 (0.00s)
--- PASS: TestNewTree_VariousSizes/#01 (0.00s)
--- PASS: TestNewTree_VariousSizes/#02 (0.00s)
--- PASS: TestNewTree_VariousSizes/#03 (0.00s)
--- PASS: TestNewTree_VariousSizes/#04 (0.00s)
--- PASS: TestNewTree_VariousSizes/#05 (0.00s)
--- PASS: TestNewTree_VariousSizes/#06 (0.00s)
=== RUN    TestNewTree_LeavesSortedByHash
--- PASS: TestNewTree_LeavesSortedByHash (0.00s)
=== RUN    TestGetProof_Verify
--- PASS: TestGetProof_Verify (0.00s)
=== RUN    TestGetProof_SingleCluster
--- PASS: TestGetProof_SingleCluster (0.00s)
=== RUN    TestGetProof_NotFound
--- PASS: TestGetProof_NotFound (0.00s)
=== RUN    TestGetProof_LargeTree
--- PASS: TestGetProof_LargeTree (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/merkle        1.243s
?      github.com/ssvlabs/ssv-oracle/observability [no test files]
=== RUN    TestNew
--- PASS: TestNew (0.00s)
=== RUN    TestOracleStorageInterface
--- PASS: TestOracleStorageInterface (0.00s)
=== RUN    TestAggregateByCluster_MultipleClusters
--- PASS: TestAggregateByCluster_MultipleClusters (0.00s)
=== RUN    TestDeduplicatePubkeys
--- PASS: TestDeduplicatePubkeys (0.00s)
=== RUN    TestEmptyValidators
--- PASS: TestEmptyValidators (0.00s)
=== RUN    TestOracleSchedule
--- PASS: TestOracleSchedule (0.00s)
=== RUN    TestAggregateByCluster_BalanceFloor
```

```
--- PASS: TestAggregateByCluster_BalanceFloor (0.00s)
=== RUN    TestAggregateByCluster_AllBelowThreshold
--- PASS: TestAggregateByCluster_AllBelowThreshold (0.00s)
=== RUN    TestAggregateByCluster_NotOnBeacon
--- PASS: TestAggregateByCluster_NotOnBeacon (0.00s)
=== RUN    TestCommitPhase_TargetAt
--- PASS: TestCommitPhase_TargetAt (0.00s)
=== RUN    TestCommitSchedule_Validate
=== RUN    TestCommitSchedule_Validate/valid_single_phase
=== RUN    TestCommitSchedule_Validate/valid_multiple_phases
=== RUN    TestCommitSchedule_Validate/empty_schedule
=== RUN    TestCommitSchedule_Validate/zero_interval
=== RUN    TestCommitSchedule_Validate/unsorted_phases
=== RUN    TestCommitSchedule_Validate/duplicate_start_epochs
=== RUN    TestCommitSchedule_Validate/misaligned_phases
=== RUN    TestCommitSchedule_Validate/aligned_phases
--- PASS: TestCommitSchedule_Validate (0.00s)
    --- PASS: TestCommitSchedule_Validate/valid_single_phase (0.00s)
    --- PASS: TestCommitSchedule_Validate/valid_multiple_phases (0.00s)
    --- PASS: TestCommitSchedule_Validate/empty_schedule (0.00s)
    --- PASS: TestCommitSchedule_Validate/zero_interval (0.00s)
    --- PASS: TestCommitSchedule_Validate/unsorted_phases (0.00s)
    --- PASS: TestCommitSchedule_Validate/duplicate_start_epochs (0.00s)
    --- PASS: TestCommitSchedule_Validate/misaligned_phases (0.00s)
    --- PASS: TestCommitSchedule_Validate/aligned_phases (0.00s)
=== RUN    TestCommitSchedule_PhaseAt
=== RUN    TestCommitSchedule_PhaseAt/before_first_phase
=== RUN    TestCommitSchedule_PhaseAt/exactly_at_first_phase_start
=== RUN    TestCommitSchedule_PhaseAt/between_phases
=== RUN    TestCommitSchedule_PhaseAt/exactly_at_second_phase_start
=== RUN    TestCommitSchedule_PhaseAt/after_second_phase
--- PASS: TestCommitSchedule_PhaseAt (0.00s)
    --- PASS: TestCommitSchedule_PhaseAt/before_first_phase (0.00s)
    --- PASS: TestCommitSchedule_PhaseAt/exactly_at_first_phase_start (0.00s)
    --- PASS: TestCommitSchedule_PhaseAt/between_phases (0.00s)
    --- PASS: TestCommitSchedule_PhaseAt/exactly_at_second_phase_start (0.00s)
    --- PASS: TestCommitSchedule_PhaseAt/after_second_phase (0.00s)
=== RUN    TestCommitSchedule_NextTarget
=== RUN    TestCommitSchedule_NextTarget/before_schedule
=== RUN    TestCommitSchedule_NextTarget/at_schedule_start
=== RUN    TestCommitSchedule_NextTarget/just_after_start
=== RUN    TestCommitSchedule_NextTarget/at_first_target
=== RUN    TestCommitSchedule_NextTarget/between_targets
=== RUN    TestCommitSchedule_NextTarget/at_second_target
--- PASS: TestCommitSchedule_NextTarget (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/before_schedule (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/at_schedule_start (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/just_after_start (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/at_first_target (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/between_targets (0.00s)
    --- PASS: TestCommitSchedule_NextTarget/at_second_target (0.00s)
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_at_140
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_at_145
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_150
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_152
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_155
=== RUN    TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_before_transition
--- PASS: TestCommitSchedule_NextTarget_PhaseTransition (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_at_140 (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_at_145 (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_150 (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_152 (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_2_-_at_155 (0.00s)
    --- PASS: TestCommitSchedule_NextTarget_PhaseTransition/phase_1_-_before_transition (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/oracle    2.183s
=== RUN    TestStorage_Connection
2026-05-17T17:06:07.614-0400 INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_Connection (0.02s)
=== RUN    TestStorage_SyncProgress
2026-05-17T17:06:07.621-0400 INFO    logger/logger.go:86 Database schema applied
```

```
--- PASS: TestStorage_SyncProgress (0.00s)
=== RUN    TestStorage_ChainID
2026-05-17T17:06:07.624-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_ChainID (0.00s)
=== RUN    TestStorage_Cluster
2026-05-17T17:06:07.628-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_Cluster (0.01s)
=== RUN    TestStorage_GetCluster_NotFound
2026-05-17T17:06:07.638-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCluster_NotFound (0.01s)
=== RUN    TestStorage_Validator
2026-05-17T17:06:07.641-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_Validator (0.00s)
=== RUN    TestStorage_ValidatorInvalidPubkey
2026-05-17T17:06:07.644-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_ValidatorInvalidPubkey (0.00s)
=== RUN    TestStorage_Transaction
2026-05-17T17:06:07.648-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_Transaction (0.00s)
=== RUN    TestStorage_TransactionRollback
2026-05-17T17:06:07.651-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_TransactionRollback (0.00s)
=== RUN    TestStorage_InsertEvent
2026-05-17T17:06:07.655-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_InsertEvent (0.00s)
=== RUN    TestStorage_OracleCommit
2026-05-17T17:06:07.660-0400    INFO    logger/logger.go:86 Database schema applied
2026-05-17T17:06:07.662-0400    WARN    logger/logger.go:101 Duplicate commit ignored
{"targetEpoch": 100}
github.com/ssvlabs/ssv-oracle/logger.Warnw
    github.com/ssvlabs/ssv-oracle/logger/logger.go:101
github.com/ssvlabs/ssv-oracle/storage.(*Storage).InsertPendingCommit
    github.com/ssvlabs/ssv-oracle/storage/storage.go:354
github.com/ssvlabs/ssv-oracle/storage.TestStorage_OracleCommit
    github.com/ssvlabs/ssv-oracle/storage/storage_test.go:467
testing.tRunner
    testing/testing.go:1934
--- PASS: TestStorage_OracleCommit (0.01s)
=== RUN    TestStorage_GetCommitByBlock_NotFound
2026-05-17T17:06:07.666-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCommitByBlock_NotFound (0.00s)
=== RUN    TestStorage_GetLatestCommit_NoCommits
2026-05-17T17:06:07.669-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetLatestCommit_NoCommits (0.00s)
=== RUN    TestStorage_GetLatestCommit_OnlyPending
2026-05-17T17:06:07.673-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetLatestCommit_OnlyPending (0.00s)
=== RUN    TestStorage_GetLatestCommit_ReturnsLatestConfirmed
2026-05-17T17:06:07.676-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetLatestCommit_ReturnsLatestConfirmed (0.00s)
=== RUN    TestStorage_ClearAllState
2026-05-17T17:06:07.679-0400    INFO    logger/logger.go:86 Database schema applied
2026-05-17T17:06:07.680-0400    INFO    logger/logger.go:86 Database cleared
--- PASS: TestStorage_ClearAllState (0.00s)
=== RUN    TestStorage_DeleteCluster_CascadesValidators
2026-05-17T17:06:07.693-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_DeleteCluster_CascadesValidators (0.01s)
=== RUN    TestStorage_WALModeEnabled
2026-05-17T17:06:07.697-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_WALModeEnabled (0.00s)
=== RUN    TestStorage_ForeignKeysEnabled
2026-05-17T17:06:07.700-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_ForeignKeysEnabled (0.00s)
=== RUN    TestStorage_GetCommitByEpoch
2026-05-17T17:06:07.704-0400    INFO    logger/logger.go:86 Database schema applied
=== RUN    TestStorage_GetCommitByEpoch/not_found
=== RUN    TestStorage_GetCommitByEpoch/first_commit_has_no_prev
=== RUN    TestStorage_GetCommitByEpoch/middle_commit_has_both
=== RUN    TestStorage_GetCommitByEpoch/last_commit_has_no_next
--- PASS: TestStorage_GetCommitByEpoch (0.01s)
    --- PASS: TestStorage_GetCommitByEpoch/not_found (0.00s)
    --- PASS: TestStorage_GetCommitByEpoch/first_commit_has_no_prev (0.00s)
```

```
--- PASS: TestStorage_GetCommitByEpoch/middle_commit_has_both (0.00s)
--- PASS: TestStorage_GetCommitByEpoch/last_commit_has_no_next (0.00s)
=== RUN    TestStorage_GetCommitByEpoch_ReturnsAllStatuses
2026-05-17T17:06:07.712-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCommitByEpoch_ReturnsAllStatuses (0.00s)
=== RUN    TestStorage_GetCommitByEpoch_PrevNextSpanAllStatuses
2026-05-17T17:06:07.716-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCommitByEpoch_PrevNextSpanAllStatuses (0.00s)
=== RUN    TestStorage_GetCommitByEpoch_SingleCommit
2026-05-17T17:06:07.719-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCommitByEpoch_SingleCommit (0.00s)
=== RUN    TestStorage_GetCommitByEpoch_ClusterBalances
2026-05-17T17:06:07.722-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetCommitByEpoch_ClusterBalances (0.00s)
=== RUN    TestStorage_GetAllClusterInfo
2026-05-17T17:06:07.726-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetAllClusterInfo (0.00s)
=== RUN    TestStorage_DecodeOperatorIDs_Malformed
=== RUN    TestStorage_DecodeOperatorIDs_Malformed/empty_string
=== RUN    TestStorage_DecodeOperatorIDs_Malformed/invalid_json
=== RUN    TestStorage_DecodeOperatorIDs_Malformed/wrong_type
=== RUN    TestStorage_DecodeOperatorIDs_Malformed/array_of_strings
=== RUN    TestStorage_DecodeOperatorIDs_Malformed/unclosed_bracket
--- PASS: TestStorage_DecodeOperatorIDs_Malformed (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Malformed/empty_string (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Malformed/invalid_json (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Malformed/wrong_type (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Malformed/array_of_strings (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Malformed/unclosed_bracket (0.00s)
=== RUN    TestStorage_DecodeOperatorIDs_Valid
=== RUN    TestStorage_DecodeOperatorIDs_Valid/empty_array
=== RUN    TestStorage_DecodeOperatorIDs_Valid/single_element
=== RUN    TestStorage_DecodeOperatorIDs_Valid/multiple_elements
=== RUN    TestStorage_DecodeOperatorIDs_Valid/with_spaces
--- PASS: TestStorage_DecodeOperatorIDs_Valid (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Valid/empty_array (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Valid/single_element (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Valid/multiple_elements (0.00s)
--- PASS: TestStorage_DecodeOperatorIDs_Valid/with_spaces (0.00s)
=== RUN    TestStorage_GetFinalizedClusters_ReturnsRequestedRowsAndLastSynced
2026-05-17T17:06:07.732-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetFinalizedClusters_ReturnsRequestedRowsAndLastSynced (0.00s)
=== RUN    TestStorage_GetFinalizedClusters_OmitsMissingIDs
2026-05-17T17:06:07.736-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetFinalizedClusters_OmitsMissingIDs (0.00s)
=== RUN    TestStorage_GetFinalizedClusters_EmptyIDsReturnsLastSynced
2026-05-17T17:06:07.740-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetFinalizedClusters_EmptyIDsReturnsLastSynced (0.00s)
=== RUN    TestStorage_GetFinalizedClusters_ReadAndSubsequentWriteSucceed
2026-05-17T17:06:07.743-0400    INFO    logger/logger.go:86 Database schema applied
--- PASS: TestStorage_GetFinalizedClusters_ReadAndSubsequentWriteSucceed (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/storage    1.944s
=== RUN    TestTxPolicy_Validate
=== RUN    TestTxPolicy_Validate/valid_policy
=== RUN    TestTxPolicy_Validate/gas_bump_too_low
=== RUN    TestTxPolicy_Validate/zero_attempts
=== RUN    TestTxPolicy_Validate/missing_max_fee
=== RUN    TestTxPolicy_Validate/gas_buffer_too_high
=== RUN    TestTxPolicy_Validate/negative_gas_buffer
=== RUN    TestTxPolicy_Validate/zero_pending_timeout
=== RUN    TestTxPolicy_Validate/negative_retry_delay
--- PASS: TestTxPolicy_Validate (0.00s)
--- PASS: TestTxPolicy_Validate/valid_policy (0.00s)
--- PASS: TestTxPolicy_Validate/gas_bump_too_low (0.00s)
--- PASS: TestTxPolicy_Validate/zero_attempts (0.00s)
--- PASS: TestTxPolicy_Validate/missing_max_fee (0.00s)
--- PASS: TestTxPolicy_Validate/gas_buffer_too_high (0.00s)
--- PASS: TestTxPolicy_Validate/negative_gas_buffer (0.00s)
--- PASS: TestTxPolicy_Validate/zero_pending_timeout (0.00s)
--- PASS: TestTxPolicy_Validate/negative_retry_delay (0.00s)
=== RUN    TestTxPolicy_ApplyDefaults
```

```
--- PASS: TestTxPolicy_ApplyDefaults (0.00s)
=== RUN    TestTxPolicy_MaxFeePerGasWei
=== RUN    TestTxPolicy_MaxFeePerGasWei/100_gwei
=== RUN    TestTxPolicy_MaxFeePerGasWei/1_gwei
=== RUN    TestTxPolicy_MaxFeePerGasWei/50_gwei
--- PASS: TestTxPolicy_MaxFeePerGasWei (0.00s)
--- PASS: TestTxPolicy_MaxFeePerGasWei/100_gwei (0.00s)
--- PASS: TestTxPolicy_MaxFeePerGasWei/1_gwei (0.00s)
--- PASS: TestTxPolicy_MaxFeePerGasWei/50_gwei (0.00s)
=== RUN    TestRevertError_Error
=== RUN    TestRevertError_Error/simulated_revert
=== RUN    TestRevertError_Error/tx_revert_with_hash
=== RUN    TestRevertError_Error/revert_without_hash
--- PASS: TestRevertError_Error (0.00s)
--- PASS: TestRevertError_Error/simulated_revert (0.00s)
--- PASS: TestRevertError_Error/tx_revert_with_hash (0.00s)
--- PASS: TestRevertError_Error/revert_without_hash (0.00s)
=== RUN    TestIsRevertError
=== RUN    TestIsRevertError/is_revert_error
=== RUN    TestIsRevertError/wrapped_revert_error
=== RUN    TestIsRevertError/not_revert_error
=== RUN    TestIsRevertError/nil_error
--- PASS: TestIsRevertError (0.00s)
--- PASS: TestIsRevertError/is_revert_error (0.00s)
--- PASS: TestIsRevertError/wrapped_revert_error (0.00s)
--- PASS: TestIsRevertError/not_revert_error (0.00s)
--- PASS: TestIsRevertError/nil_error (0.00s)
=== RUN    TestTxManager_DecodeErrorSelector
=== RUN    TestTxManager_DecodeErrorSelector/known_selector
=== RUN    TestTxManager_DecodeErrorSelector/another_known_selector
=== RUN    TestTxManager_DecodeErrorSelector/unknown_selector
=== RUN    TestTxManager_DecodeErrorSelector/not_a_selector_-_plain_text
=== RUN    TestTxManager_DecodeErrorSelector/too_short_hex
=== RUN    TestTxManager_DecodeErrorSelector/just_prefix
=== RUN    TestTxManager_DecodeErrorSelector/empty_string
=== RUN    TestTxManager_DecodeErrorSelector/selector_with_extra_data
--- PASS: TestTxManager_DecodeErrorSelector (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/known_selector (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/another_known_selector (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/unknown_selector (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/not_a_selector_-_plain_text (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/too_short_hex (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/just_prefix (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/empty_string (0.00s)
--- PASS: TestTxManager_DecodeErrorSelector/selector_with_extra_data (0.00s)
=== RUN    TestTxManager_ExtractRevertReason
=== RUN    TestTxManager_ExtractRevertReason/standard_revert_message
=== RUN    TestTxManager_ExtractRevertReason/known_error_selector_in_message
=== RUN    TestTxManager_ExtractRevertReason/unknown_error
=== RUN    TestTxManager_ExtractRevertReason/empty_revert_reason
=== RUN    TestTxManager_ExtractRevertReason/revert_with_spaces
--- PASS: TestTxManager_ExtractRevertReason (0.00s)
--- PASS: TestTxManager_ExtractRevertReason/standard_revert_message (0.00s)
--- PASS: TestTxManager_ExtractRevertReason/known_error_selector_in_message (0.00s)
--- PASS: TestTxManager_ExtractRevertReason/unknown_error (0.00s)
--- PASS: TestTxManager_ExtractRevertReason/empty_revert_reason (0.00s)
--- PASS: TestTxManager_ExtractRevertReason/revert_with_spaces (0.00s)
=== RUN    TestTxManager_SetErrorSelectors
--- PASS: TestTxManager_SetErrorSelectors (0.00s)
=== RUN    TestIsInsufficientFunds
=== RUN    TestIsInsufficientFunds/nil_error
=== RUN    TestIsInsufficientFunds/insufficient_funds
=== RUN    TestIsInsufficientFunds/insufficient_balance
=== RUN    TestIsInsufficientFunds/uppercase
=== RUN    TestIsInsufficientFunds/mixed_case
=== RUN    TestIsInsufficientFunds/unrelated_error
=== RUN    TestIsInsufficientFunds/empty_error
--- PASS: TestIsInsufficientFunds (0.00s)
--- PASS: TestIsInsufficientFunds/nil_error (0.00s)
--- PASS: TestIsInsufficientFunds/insufficient_funds (0.00s)
--- PASS: TestIsInsufficientFunds/insufficient_balance (0.00s)
--- PASS: TestIsInsufficientFunds/uppercase (0.00s)
```

```
--- PASS: TestIsInsufficientFunds/mixed_case (0.00s)
--- PASS: TestIsInsufficientFunds/unrelated_error (0.00s)
--- PASS: TestIsInsufficientFunds/empty_error (0.00s)
=== RUN    TestIsNonceTooLow
=== RUN    TestIsNonceTooLow/nil_error
=== RUN    TestIsNonceTooLow/nonce_too_low
=== RUN    TestIsNonceTooLow/uppercase
=== RUN    TestIsNonceTooLow/mixed_case
=== RUN    TestIsNonceTooLow/unrelated_error
=== RUN    TestIsNonceTooLow/empty_error
--- PASS: TestIsNonceTooLow (0.00s)
--- PASS: TestIsNonceTooLow/nil_error (0.00s)
--- PASS: TestIsNonceTooLow/nonce_too_low (0.00s)
--- PASS: TestIsNonceTooLow/uppercase (0.00s)
--- PASS: TestIsNonceTooLow/mixed_case (0.00s)
--- PASS: TestIsNonceTooLow/unrelated_error (0.00s)
--- PASS: TestIsNonceTooLow/empty_error (0.00s)
=== RUN    TestIsTxAlreadyKnown
=== RUN    TestIsTxAlreadyKnown/nil_error
=== RUN    TestIsTxAlreadyKnown/already_known
=== RUN    TestIsTxAlreadyKnown/uppercase
=== RUN    TestIsTxAlreadyKnown/in_context
=== RUN    TestIsTxAlreadyKnown/unrelated_error
=== RUN    TestIsTxAlreadyKnown/empty_error
--- PASS: TestIsTxAlreadyKnown (0.00s)
--- PASS: TestIsTxAlreadyKnown/nil_error (0.00s)
--- PASS: TestIsTxAlreadyKnown/already_known (0.00s)
--- PASS: TestIsTxAlreadyKnown/uppercase (0.00s)
--- PASS: TestIsTxAlreadyKnown/in_context (0.00s)
--- PASS: TestIsTxAlreadyKnown/unrelated_error (0.00s)
--- PASS: TestIsTxAlreadyKnown/empty_error (0.00s)
=== RUN    TestIsUnderpriced
=== RUN    TestIsUnderpriced/nil_error
=== RUN    TestIsUnderpriced/replacement_underpriced
=== RUN    TestIsUnderpriced/max_fee_too_low
=== RUN    TestIsUnderpriced/generic_underpriced
=== RUN    TestIsUnderpriced/uppercase
=== RUN    TestIsUnderpriced/unrelated_error
=== RUN    TestIsUnderpriced/empty_error
--- PASS: TestIsUnderpriced (0.00s)
--- PASS: TestIsUnderpriced/nil_error (0.00s)
--- PASS: TestIsUnderpriced/replacement_underpriced (0.00s)
--- PASS: TestIsUnderpriced/max_fee_too_low (0.00s)
--- PASS: TestIsUnderpriced/generic_underpriced (0.00s)
--- PASS: TestIsUnderpriced/uppercase (0.00s)
--- PASS: TestIsUnderpriced/unrelated_error (0.00s)
--- PASS: TestIsUnderpriced/empty_error (0.00s)
=== RUN    TestIsContractRevert
=== RUN    TestIsContractRevert/nil_error
=== RUN    TestIsContractRevert/execution_reverted
=== RUN    TestIsContractRevert/uppercase_execution_reverted
=== RUN    TestIsContractRevert/execution_reverted_no_reason
=== RUN    TestIsContractRevert/network_error
=== RUN    TestIsContractRevert/timeout_error
=== RUN    TestIsContractRevert/rpc_error
=== RUN    TestIsContractRevert/empty_error
--- PASS: TestIsContractRevert (0.00s)
--- PASS: TestIsContractRevert/nil_error (0.00s)
--- PASS: TestIsContractRevert/execution_reverted (0.00s)
--- PASS: TestIsContractRevert/uppercase_execution_reverted (0.00s)
--- PASS: TestIsContractRevert/execution_reverted_no_reason (0.00s)
--- PASS: TestIsContractRevert/network_error (0.00s)
--- PASS: TestIsContractRevert/timeout_error (0.00s)
--- PASS: TestIsContractRevert/rpc_error (0.00s)
--- PASS: TestIsContractRevert/empty_error (0.00s)
=== RUN    TestErrorTypes
=== RUN    TestErrorTypes/errBaseFeeExceedsMax_is_distinguishable
=== RUN    TestErrorTypes/all_error_types_are_distinct
--- PASS: TestErrorTypes (0.00s)
--- PASS: TestErrorTypes/errBaseFeeExceedsMax_is_distinguishable (0.00s)
--- PASS: TestErrorTypes/all_error_types_are_distinct (0.00s)
PASS
```

```
ok      github.com/ssvlabs/ssv-oracle/txmanager 2.529s
=== RUN   TestProcessCommit_EmptyClusters
2026-05-17T17:06:08.976-0400      INFO      updater/updater.go:215  No clusters to update   {"blockNum": 1000, "targetEpoch": 100}
--- PASS: TestProcessCommit_EmptyClusters (0.00s)
=== RUN   TestProcessCommit_RootMismatch
2026-05-17T17:06:08.978-0400      DEBUG     updater/updater.go:220  Merkle tree built   {"blockNum": 1000, "targetEpoch": 100, "root": "0x6aba4a4925ba0836303c53cb8fbf4a34bc84e70078efef15cfd6e1854f55e7f0", "clusters": 1}
--- PASS: TestProcessCommit_RootMismatch (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/updater 2.778s
=== RUN   TestEnvSigner
--- PASS: TestEnvSigner (0.00s)
=== RUN   TestEnvSigner_MissingEnvVar
--- PASS: TestEnvSigner_MissingEnvVar (0.00s)
=== RUN   TestEnvSigner_EmptyEnvVarName
--- PASS: TestEnvSigner_EmptyEnvVarName (0.00s)
=== RUN   TestEnvSigner_InvalidPrivateKey
--- PASS: TestEnvSigner_InvalidPrivateKey (0.00s)
=== RUN   TestKeystoreSigner
--- PASS: TestKeystoreSigner (2.28s)
=== RUN   TestKeystoreSigner_PasswordFromFile
--- PASS: TestKeystoreSigner_PasswordFromFile (2.06s)
=== RUN   TestKeystoreSigner_WrongPassword
--- PASS: TestKeystoreSigner_WrongPassword (2.06s)
=== RUN   TestFactory
=== RUN   TestFactory/env_signer
=== RUN   TestFactory/missing_type
=== RUN   TestFactory/env_signer_missing_key
=== RUN   TestFactory/keystore_missing_path
=== RUN   TestFactory/keystore_missing_password
=== RUN   TestFactory/unsupported_type
=== RUN   TestFactory/nil_config
--- PASS: TestFactory (0.00s)
--- PASS: TestFactory/env_signer (0.00s)
--- PASS: TestFactory/missing_type (0.00s)
--- PASS: TestFactory/env_signer_missing_key (0.00s)
--- PASS: TestFactory/keystore_missing_path (0.00s)
--- PASS: TestFactory/keystore_missing_password (0.00s)
--- PASS: TestFactory/unsupported_type (0.00s)
--- PASS: TestFactory/nil_config (0.00s)
=== RUN   TestSignerClose
--- PASS: TestSignerClose (0.00s)
PASS
ok      github.com/ssvlabs/ssv-oracle/wallet 8.745s
```

Code Coverage

Coverage on the six in-scope files is 29.7% (19 of 64 functions) and is concentrated on deterministic, in-memory logic — merkle/tree.go is 90% covered, and oracle.go's pure helpers (aggregateByCluster , deduplicatePubkeys) are at 100%. Everything that crosses a process boundary is untested: contract/client.go is 0% across all six functions, eth/syncer/syncer.go is 9.5% (only New and computeClusterIDFromEvent), and 7 of 10 event parsers in parser.go are 0%. The two consensus-critical pipelines — the commit cycle (oracle.commit → contract.CommitRoot) and the event sync-and-apply pipeline (syncer.SyncToBlock → parser.parse* → syncer.handle*) — have zero end-to-end coverage, as does the entire updater. Run loop. A regression in any event handler, revert classification, or post-finalization apply order would not be caught by the existing suite.

Fix review 2 (commit 3c1f874): The test coverage has increased to 31.7% (20 of 63 functions) across the six in-scope files.

Package	File	Line	Function	Coverage
---	---	---:	---	---:
api	api.go	39	New	100.0%
api	api.go	47	Run	0.0%

Package	File	Line	Function	Coverage
api	api.go	89	contentTypeMiddlew are	100.0%
api	api.go	96	recoveryMiddlew are	100.0%
api	api.go	109	writeJSON	66.7%
api	api.go	116	writeError	100.0%
api	api.go	120	buildTree	100.0%
api	handlers.go	24	handleGetCom mit	95.2%
api	handlers.go	104	handleGetProo f	100.0%
api	handlers.go	156	handleUI	0.0%
api	handlers.go	168	resolveCommit	93.3%
api	handlers.go	195	handleResolve Error	100.0%
api	handlers.go	208	parseClusterID	100.0%
api	handlers.go	222	toHex	100.0%
api	handlers.go	227	toHexOrEmpty	100.0%
api	handlers.go	235	rootMatches	100.0%
api	handlers.go	239	computeDiff	100.0%
commands	config.go	50	getMEVRPCs	100.0%
commands	config.go	65	validate	86.7%
commands	config.go	115	validateAddres s	100.0%
commands	config.go	129	validateURL	92.3%
commands	config.go	154	loadConfig	100.0%
commands	root.go	25	Execute	0.0%
commands	root.go	29	init	100.0%
commands	run.go	48	init	100.0%
commands	run.go	54	run	0.0%
commands	run.go	167	validateChainI D	0.0%
commands	run.go	193	runServices	0.0%

Package	File	Line	Function	Coverage
commands	run.go	243	runMetricsServer	0.0%
oracle	main.go	20	main	0.0%
contract	abi.go	25	init	81.8%
contract	client.go	51	NewClient	0.0%
contract	client.go	93	Close	0.0%
contract	client.go	107	CommitRoot	0.0%
contract	client.go	121	GetCommittedRoot	0.0%
contract	client.go	144	GetClusterEffectiveBalance	0.0%
contract	client.go	174	UpdateClusterBalance	0.0%
contract	events.go	27	SubscribeRootCommitted	0.0%
contract	events.go	59	processRootCommittedLogs	0.0%
contract	events.go	104	parseRootCommittedEvent	0.0%
beacon	client.go	58	New	0.0%
beacon	client.go	92	GetFinalizedEpoch	0.0%
beacon	client.go	107	fetchCheckpoint	0.0%
beacon	client.go	129	VerifyFinalizedBlockRoot	0.0%
beacon	client.go	148	GetValidatorBalances	0.0%
beacon	client.go	204	SubscribeFinalizedCheckpoints	0.0%
beacon	client.go	251	IsRetriable	100.0%
execution	batch_size.go	22	NewAdaptiveBatchSize	100.0%
execution	batch_size.go	40	Get	100.0%
execution	batch_size.go	48	Decrease	100.0%
execution	batch_size.go	61	Increase	90.9%

Package	File	Line	Function	Coverage
execution	client.go	63	New	0.0%
execution	client.go	96	Close	0.0%
execution	client.go	101	GetFinalizedBlock	0.0%
execution	client.go	115	GetHeadBlock	0.0%
execution	client.go	131	FetchLogs	0.0%
execution	client.go	188	categorizeBatchError	100.0%
execution	client.go	233	packLogs	0.0%
execution	client.go	258	groupLogsByBlock	100.0%
execution	client.go	294	getBlockTimestampsBatch	0.0%
execution	client.go	335	fetchLogsBatch	0.0%
execution	client.go	365	GetChainID	0.0%
eth	retry.go	26	DefaultRetryConfig	0.0%
eth	retry.go	37	WithRetry	94.1%
syncer	headstate.go	25	BuildHeadStateSnapshot	0.0%
syncer	headstate.go	91	applyClusterEventToOverlay	100.0%
syncer	parser.go	23	newParser	100.0%
syncer	parser.go	27	parseLog	33.3%
syncer	parser.go	64	parseValidatorAdded	23.1%
syncer	parser.go	92	parseValidatorRemoved	0.0%
syncer	parser.go	118	parseClusterLiquidated	0.0%
syncer	parser.go	142	parseClusterReactivated	0.0%
syncer	parser.go	166	parseClusterWithdrawn	0.0%
syncer	parser.go	192	parseClusterDeposited	0.0%

Package	File	Line	Function	Coverage
syncer	parser.go	218	parseClusterM igratedToETH	0.0%
syncer	parser.go	248	parseClusterB alanceUpdate d	0.0%
syncer	syncer.go	45	New	100.0%
syncer	syncer.go	55	SyncToFinalize d	0.0%
syncer	syncer.go	91	SyncToBlock	0.0%
syncer	syncer.go	179	runFetcher	0.0%
syncer	syncer.go	202	processBatch	0.0%
syncer	syncer.go	248	processLog	0.0%
syncer	syncer.go	282	storeRawEvent	0.0%
syncer	syncer.go	306	applyEvent	0.0%
syncer	syncer.go	329	handleValidato rAdded	0.0%
syncer	syncer.go	347	handleValidato rRemoved	0.0%
syncer	syncer.go	354	handleClusterL iquidated	0.0%
syncer	syncer.go	358	handleCluster Reactivated	0.0%
syncer	syncer.go	362	handleCluster Withdrawn	0.0%
syncer	syncer.go	366	handleCluster Deposited	0.0%
syncer	syncer.go	370	handleCluster MigratedToET H	0.0%
syncer	syncer.go	374	handleCluster BalanceUpdat ed	0.0%
syncer	syncer.go	378	upsertClusterF romEvent	0.0%
syncer	syncer.go	392	computeClust erIDFromEvent	100.0%
syncer	topics.go	26	EventTopics	100.0%
syncer	types.go	101	computeClust erID	100.0%

Package	File	Line	Function	Coverage
syncer	types.go	131	clusterKey	100.0%
syncer	types.go	132	cluster	0.0%
syncer	types.go	134	clusterKey	100.0%
syncer	types.go	137	cluster	0.0%
syncer	types.go	139	clusterKey	100.0%
syncer	types.go	142	cluster	100.0%
syncer	types.go	144	clusterKey	100.0%
syncer	types.go	147	cluster	100.0%
syncer	types.go	149	clusterKey	100.0%
syncer	types.go	152	cluster	0.0%
syncer	types.go	154	clusterKey	100.0%
syncer	types.go	157	cluster	0.0%
syncer	types.go	159	clusterKey	100.0%
syncer	types.go	162	cluster	0.0%
syncer	types.go	164	clusterKey	100.0%
syncer	types.go	167	cluster	0.0%
logger	logger.go	16	init	100.0%
logger	logger.go	24	Init	0.0%
logger	logger.go	65	Sync	0.0%
logger	logger.go	72	With	100.0%
logger	logger.go	77	Debug	0.0%
logger	logger.go	80	Debugf	0.0%
logger	logger.go	83	Debugw	100.0%
logger	logger.go	86	Info	100.0%
logger	logger.go	89	Infof	0.0%
logger	logger.go	92	Infow	0.0%
logger	logger.go	95	Warn	0.0%
logger	logger.go	98	Warnf	0.0%
logger	logger.go	101	Warnw	100.0%

Package	File	Line	Function	Coverage
logger	logger.go	104	Error	0.0%
logger	logger.go	107	Errorf	0.0%
logger	logger.go	110	Errorw	100.0%
logger	logger.go	113	Fatal	0.0%
logger	logger.go	116	Fatalf	0.0%
logger	logger.go	119	Fatalw	0.0%
merkle	encoding.go	10	init	100.0%
merkle	encoding.go	19	HashLeaf	75.0%
merkle	encoding.go	27	mustType	75.0%
merkle	tree.go	26	Leaves	100.0%
merkle	tree.go	35	InnerLayers	100.0%
merkle	tree.go	46	NewTree	100.0%
merkle	tree.go	69	GetProof	100.0%
merkle	tree.go	86	findLeaf	100.0%
merkle	tree.go	95	sibling	100.0%
merkle	tree.go	106	makeLeaves	100.0%
merkle	tree.go	118	buildLayers	100.0%
merkle	tree.go	132	hashLevel	100.0%
merkle	tree.go	144	hashPair	100.0%
observability	observability.go	20	Setup	0.0%
observability	observability.go	37	NewMetric	66.7%
oracle	observability.go	52	recordCommit	0.0%
oracle	observability.go	56	recordNextTarget	0.0%
oracle	observability.go	60	recordCommitStats	0.0%
oracle	oracle.go	56	New	100.0%
oracle	oracle.go	67	Run	0.0%
oracle	oracle.go	152	commit	0.0%

Package	File	Line	Function	Coverage
oracle	oracle.go	239	buildTree	0.0%
oracle	oracle.go	249	fetchBalances	0.0%
oracle	oracle.go	275	deduplicatePu bkeys	100.0%
oracle	oracle.go	289	aggregateByCl uster	100.0%
oracle	oracle.go	323	handleCommit Error	0.0%
oracle	phase.go	12	TargetAt	100.0%
oracle	phase.go	20	Validate	100.0%
oracle	phase.go	45	PhaseAt	100.0%
oracle	phase.go	56	NextTarget	100.0%
storage	storage.go	122	New	73.7%
storage	storage.go	167	Close	100.0%
storage	storage.go	172	GetChainID	80.0%
storage	storage.go	182	SetChainID	75.0%
storage	storage.go	191	GetLastSynce dBlock	80.0%
storage	storage.go	201	UpdateLastSy ncedBlock	100.0%
storage	storage.go	206	InsertEvent	100.0%
storage	storage.go	211	UpsertCluster	100.0%
storage	storage.go	216	DeleteCluster	100.0%
storage	storage.go	221	GetCluster	100.0%
storage	storage.go	233	GetFinalizedCl usters	81.2%
storage	storage.go	264	getClusterRow	84.2%
storage	storage.go	302	InsertValidator	100.0%
storage	storage.go	307	DeleteValidato r	100.0%
storage	storage.go	312	GetActiveValid ators	84.6%
storage	storage.go	338	InsertPending Commit	80.0%

Package	File	Line	Function	Coverage
storage	storage.go	361	UpdateCommitStatus	75.0%
storage	storage.go	374	GetCommitByBlock	85.7%
storage	storage.go	400	GetLatestCommit	84.6%
storage	storage.go	429	GetCommitByEpoch	86.7%
storage	storage.go	460	GetAllClusterInfo	81.2%
storage	storage.go	487	ClearAllState	70.6%
storage	storage.go	519	BeginTx	75.0%
storage	storage.go	533	Commit	100.0%
storage	storage.go	536	Rollback	100.0%
storage	storage.go	539	InsertEvent	0.0%
storage	storage.go	544	UpsertCluster	100.0%
storage	storage.go	549	DeleteCluster	0.0%
storage	storage.go	554	InsertValidator	100.0%
storage	storage.go	559	DeleteValidator	0.0%
storage	storage.go	564	UpdateLastSyncedBlock	0.0%
storage	storage.go	568	insertEvent	80.0%
storage	storage.go	587	upsertCluster	75.0%
storage	storage.go	618	deleteCluster	75.0%
storage	storage.go	626	insertValidator	83.3%
storage	storage.go	640	deleteValidator	75.0%
storage	storage.go	651	updateLastSyncedBlock	75.0%
storage	storage.go	659	boolToInt	100.0%
storage	storage.go	666	intToBool	100.0%
storage	storage.go	670	encodeOperatorsIDs	75.0%
storage	storage.go	678	decodeOperatorsIDs	100.0%

Package	File	Line	Function	Coverage
storage	storage.go	687	SetSyncMode	0.0%
txmanager	config.go	32	ApplyDefaults	100.0%
txmanager	config.go	54	MaxFeePerGasWei	100.0%
txmanager	config.go	61	Validate	100.0%
txmanager	manager.go	59	Error	100.0%
txmanager	manager.go	91	New	0.0%
txmanager	manager.go	128	SetErrorSelectors	100.0%
txmanager	manager.go	133	Close	0.0%
txmanager	manager.go	140	IsRevertError	100.0%
txmanager	manager.go	152	SendTransaction	0.0%
txmanager	manager.go	334	buildAndSignTx	0.0%
txmanager	manager.go	355	sendToMEVRPCs	0.0%
txmanager	manager.go	393	applyBumpPercent	0.0%
txmanager	manager.go	401	bumpOrResuggest	0.0%
txmanager	manager.go	430	cancelTx	0.0%
txmanager	manager.go	476	estimateGas	0.0%
txmanager	manager.go	507	handleReceipt	0.0%
txmanager	manager.go	525	replayForRevertReason	0.0%
txmanager	manager.go	544	suggestGasFees	0.0%
txmanager	manager.go	585	findMinedReceipt	0.0%
txmanager	manager.go	601	waitForReceipt	0.0%
txmanager	manager.go	642	ExtractRevertReason	58.8%
txmanager	manager.go	673	decodeErrorSelector	100.0%
txmanager	manager.go	684	IsContractRevert	80.0%

Package	File	Line	Function	Coverage
txmanager	manager.go	694	isTxAlreadyKnown	100.0%
txmanager	manager.go	698	isNonceTooLow	100.0%
txmanager	manager.go	702	isInsufficientFunds	100.0%
txmanager	manager.go	711	isUnderpriced	100.0%
txmanager	observability.go	27	recordTx	0.0%
txmanager	observability.go	31	recordTxReceipt	0.0%
updater	observability.go	26	recordClusterUpdates	0.0%
updater	updater.go	62	storageLookup	0.0%
updater	updater.go	78	overlayLookup	0.0%
updater	updater.go	94	New	0.0%
updater	updater.go	103	Run	0.0%
updater	updater.go	122	subscribeAndProcess	0.0%
updater	updater.go	159	handleEvent	0.0%
updater	updater.go	210	processCommit	25.7%
updater	updater.go	276	buildTree	100.0%
updater	updater.go	286	processAllClusters	0.0%
updater	updater.go	336	toContractCluster	0.0%
updater	updater.go	346	processCluster	0.0%
wallet	env.go	26	newEnvSigner	100.0%
wallet	env.go	48	Address	100.0%
wallet	env.go	53	Close	100.0%
wallet	env.go	62	Sign	60.0%
wallet	factory.go	7	NewSigner	92.3%
wallet	keystore.go	21	newKeystoreSigner	75.0%
wallet	keystore.go	48	Address	100.0%

Package	File	Line	Function	Coverage
wallet	keystore.go	53	Close	100.0%
wallet	keystore.go	62	Sign	60.0%
wallet	keystore.go	72	readPassword	72.7%

Changelog

- 2026-04-10 - Initial report
- 2026-05-06 - Fix-review report
- 2026-05-18 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE

THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

